UNIT-I

Introduction to Raspberry Pi

What is Raspberry Pi?

It is basically micro sized computer or commonly in common terms it is said as a computer in your palm more specifically it is a single board computer which is very low cost device and which is very easy to access.

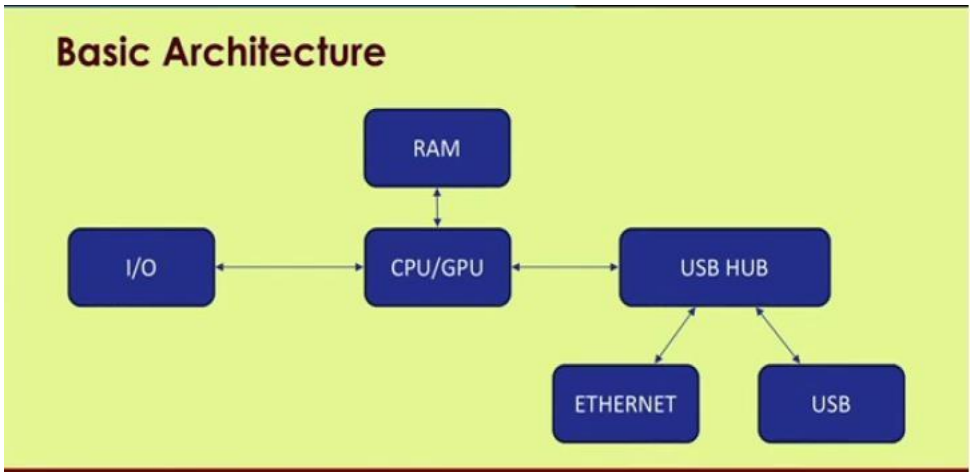
- Raspberry pi compared to Arduino is more powerful, it is more powerful in terms of the computation or processing power.
- Additionally it has better memory capacity and also it can integrate different types of sensors and actuators and this part is more attractive than compared to the similar kind of feature of Arduino.
- We can have different types of sensors integration in Raspberry pi and due to the feature that it can process more compared to Arduino it has better processing capabilities and more features and so on.
- We can configure Raspberry pi as a web server, as an edge device and so on.

Specifications

Key features	Raspberry pi 3 model B	Raspberry pi 2 model B	Raspberry Pi zero
RAM	1GB SDRAM	1GB SDRAM	512 MB SDRAM
CPU	Quad cortex A53@1.2GHz	Quad cortex A53@900MHz	ARM 11@ 1GHz
GPU	400 MHz video core IV	250 MHz video core IV	250 MHz video core IV
Ethernet	10/100	10/100	None
Wireless	802.11/Bluetooth 4.0	None	None
Video output	HDMI/Composite	HDMI/Composite	HDMI/Composite
GPIO	40	40	40

There is a provision for Wi-Fi and Bluetooth only on mod pi 3.
Generally video output is from HDMI port and there are 40 GPIO pins. So, these GPIO pins are mainly known as general purpose input output pins.

S.No.	AURDINO	RASPBERRY PI
1.	Arduino is a microcontroller, which is a part of the computer. It runs only one program again and again.	It is a mini computer with Raspbian OS. It can run multiple programs at a time.
2.	Arduino requires external hardware to connect to the internet	Raspberry Pi can be easily connected to the internet using Ethernet port and USB Wi-Fi dongles
3.	Arduino has only one USB port to connect to the computer.	Raspberry Pi has 4 USB ports to connect different devices.
4.	Processor used in Arduino is from AVR family Atmega328P	The processor used is from ARM family.
5.	Arduino uses Arduino, C/C++ type of coding	The Recommended programming language is python but C, C++, Python, ruby are pre-installed.
6.	Arduino can provide onboard storage but limted memory.	Raspberry Pi did not have storage on board. It provides an SD card port.
7.	NO inbuilt wifi, bluetooth facility	Inbuilt wifi, bluetooth facility
8.	No on board port for camera	onboard port for camera
9	Low Cost	Relatively high cost



This is the basic functional architecture of a raspberry pi.
the center you have a CPU or GPU you have various input output ports connected to it you have a RAM you have a USB hub from which you can connect an Ethernet as well as you have various USB ports to which you can connect regular USB devices.

Raspberry Pi3 b+

- Raspberry Pi is a small single board computer. By connecting peripherals like Keyboard, mouse, display to the Raspberry Pi, it will act as a mini personal computer.
- Raspberry Pi is popularly used for real time Image/Video Processing; IoT based applications and Robotics applications.
- Raspberry Pi is slower than laptop or desktop but is still a computer which can provide all the expected features or abilities, at low power consumption.
- Raspberry Pi Foundation officially provides Debian based Raspbian OS. Also, they provide NOOBS OS for Raspberry Pi. We can install several Third-Party versions of OS like Ubuntu, Archlinux, RISC OS, Windows 10 IOT Core, etc.
- Raspbian OS is official Operating System available for free to use. This OS is efficiently optimized to use with Raspberry Pi.
- Raspbian have GUI which includes tools for Browsing, Python programming, office, games, etc.
- We should use SD card (minimum 16 GB recommended) to store the OS (operating System).
- Raspberry Pi is more than computer as it provides access to the on-chip hardware i.e. GPIOs for developing an application. By accessing GPIO, we can connect devices like LED, motors, sensors, etc and can control them too.
- It has ARM based Broadcom Processor SoC along with on-chip GPU (Graphics Processing Unit).
- The CPU speed of Raspberry Pi varies from 700 MHz to 1.2 GHz. Also, it has on-board SDRAM that ranges from 256 MB to 1 GB.
- Raspberry Pi also provides on-chip SPI, I2C, and UART modules.

There are different versions of raspberry pi available as listed below:

1. **Raspberry Pi 1 Model A**

2. **Raspberry Pi 1 Model A+**
3. **Raspberry Pi 1 Model B**
4. **Raspberry Pi 1 Model B+**
5. **Raspberry Pi 2 Model B**
6. **Raspberry Pi 3 Model B**
7. **Raspberry Pi Zero**

Out of the above versions of Raspberry Pi, more prominently use Raspberry Pi and their features are as follows:

Features	Raspberry Pi Model B+	Raspberry Pi 2 Model B	Raspberry Pi 3 Model B	Raspberry Pi zero
SoC	BCM2835	BCM2836	BCM2837	BCM2835
CPU	ARM11	Quad Cortex A7	Quad Cortex A53	ARM11
Operating Freq.	700 MHz	900 MHz	1.2 GHz	1 GHz
RAM	512 MB SDRAM	1 GB SDRAM	1 GB SDRAM	512 MB SDRAM
GPU	250 MHz Videocore IV	250MHz Videocore IV	400 MHz Videocore IV	250MHz Videocore IV
Storage	micro-SD	Micro-SD	micro-SD	micro-SD
Ethernet	Yes	Yes	Yes	No
Wireless	WiFi and Bluetooth	No	No	No

Raspberry Pi 3 Technical Specifications

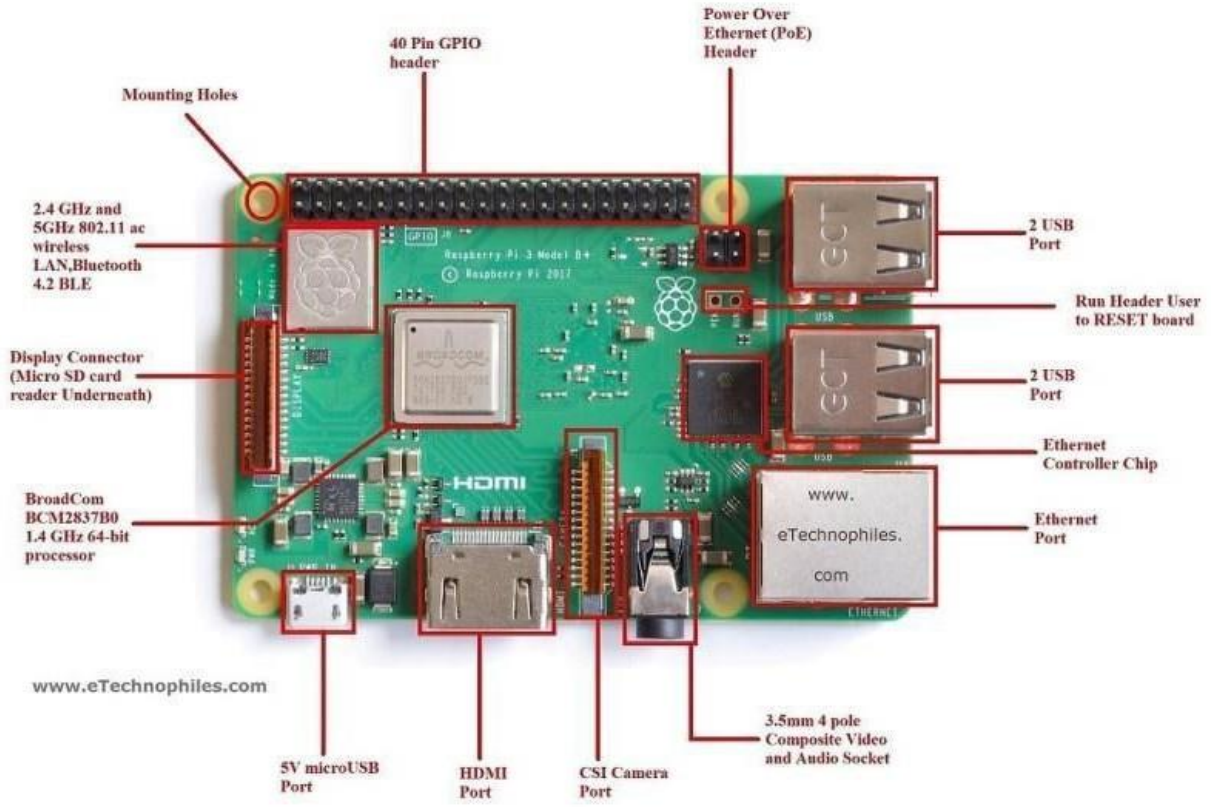
Microprocessor	Broadcom BCM2837 64bit Quad Core Processor
Processor Operating Voltage	3.3V
Raw Voltage input	5V, 2A power source
Maximum current through each I/O pin	16Ma
Maximum total current drawn from all I/O pins	54mA
Flash Memory (Operating System)	16Gbytes SSD memory card
Internal RAM	1Gbytes DDR2

Clock Frequency	1.2GHz
GPU	Dual Core Video Core IV® Multimedia Co-Processor. Provides Open GLES 2.0, hardware-accelerated Open VG, and 1080p30 H.264 high-profile decode. Capable of 1Gpixel/s, 1.5Gtexel/s or 24GFLOPs with texture filtering and DMA infrastructure.
Ethernet	10/100 Ethernet
Wireless Connectivity	BCM43143 (802.11 b/g/n Wireless LAN and Bluetooth 4.1)
Operating Temperature	-40ºC to +85ºC

Board Connectors

Name	Description
Ethernet	Base T Ethernet Socket
USB	2.0 (Four sockets)
Audio Output	3.5mm Jack and HDMI
Video output	HDMI
Camera Connector	15-pin MIPI Camera Serial Interface (CSI-2)
Display Connector	Display Serial Interface (DSI) 15 way flat flex cable connector with two data lanes and a clock lane.
Memory Card Slot	Push/Pull Micro SDIO

Raspberry Pi 3 model B Board Description



Processor & RAM: Raspberry Pi is based on an ARM processor. **64** bit ARMv7 Broadcom BCM2837 Quad Core Computer running at 1.2GHz ,

USB Ports: Raspberry Pi comes with FOUR 2.0 ports. The USB ports on Raspberry Pi can provide a current upto 100mA.
Ethernet Ports: Raspberry Pi comes with a standard RJ45 Ethernet port. We can connect an Ethernet cable or a USB WiFi adapter to provide Internet connectivity.

HDMI Output: The HDMI port on Raspberry Pi provides both video and audio output. We can connect the Raspberry Pi to a monitor using an HDMI cable. For monitors that have a DVI(Digital Video interface) port but no HDMI port, we can use an HDMI to DVI adapter/cable.

Composite Video Output: Raspberry Pi comes with a composite video output with an RCA jack that supports both PAL and NTSC video output. The RCA jack can be used to connect old televisions that have an RCA input only.

Audio Output: Raspberry Pi hasa 3.5 mm audio output jack. This audio jack is used for providing audio output to old televisions along with the RCA jack for video. The audio quality from this jack is inferior to the HDMI output.

GPIO Pins: Raspberry Pi comes with a number of general purpose input/output pins.
Fig 3.3 shows the Raspberry Pi GPIO headers. There are four types of pins on Raspberry Pi- true GPIO pins, I2C interface pins, SPI interface pins and serial Rx and Tx pins.

Display Serial Interface(DSI): The DSI interface can be used to connect an LCD panel to Raspberry Pi.

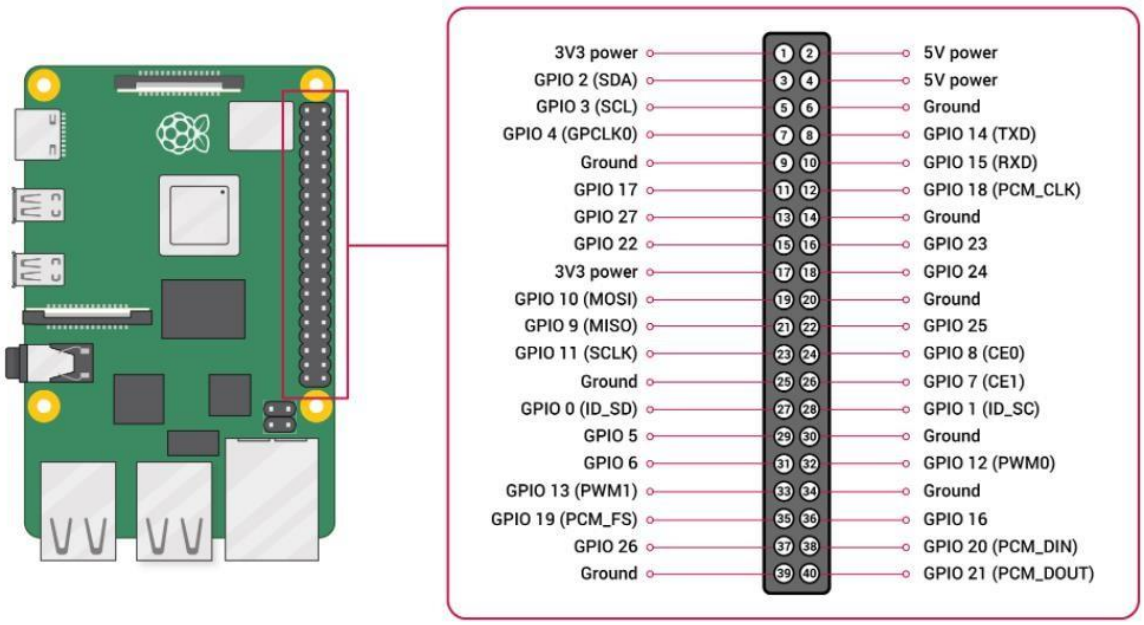
Camera Serial Interface (CSI): This interface can be used to connect a camera module to Raspberry Pi.

Status LEDs: Raspberry Pi has five status LEDs.

ACT	SD card access
PWR	3.3V Power is present
FDX	Full duplex LAN connected
LNK	Link/Network activity
100	100 Mbit LAN connected

SD Card Slot: Raspberry Pi does not have a built in operating system and storage. **SD card slot** on the **Raspberry Pi** 3 is located just below the Display Serial Adapter on the other side. We can plug-in an SD card loaded with a Linux image to the SD card slot.

Power Input: Raspberry Pi has a micro-USB connector for power input.



We require a external monitor you will require an HDMI cable to connect the monitor and the Raspberry pi you will require a keyboard and mouse a basic 5 volt adapter to power up the pi LAN cable and your memory card which will include the operating system on it.

Raspberry Pi on board Serial Communication standards

UART

UART is multi master communication protocol. This protocol is quite easy to use and very convenient for communicating between several boards: Raspberry Pi to Raspberry Pi, or Raspberry Pi to Arduino, etc.

For using UART you need 3 pins:

- GND that will connect to the global GND of your circuit.
- RX for Reception. It will connect this pin to the TX pin of the other component.
- TX for Transmission. It will connect this pin to the RX of the other component.

If the component you're communicating with is not already powered, you'll also have to use a power pin (3.3V or 5V) to power on that component.

By using a UART to USB converter, you can communicate between your laptop and Raspberry Pi with UART.

I2C

I2C is a 2 WIRE master-slave bus protocol (it can have multiple masters but mostly used with one master and multiple slaves). The most common use of I2C is to read data from sensors and actuate some components.

The master is the Raspberry Pi, and the slaves are all connected to the same bus. Each slave has a unique ID, so the Raspberry Pi knows which component it should talk to.

For using I2C you'll need 3 pins:

- GND:
- SCL: clock of the I2C. Connect all the slaves SCL to the SCL bus.
- SDA: exchanged data. Connect all the slaves SDA to the SDA bus.
-

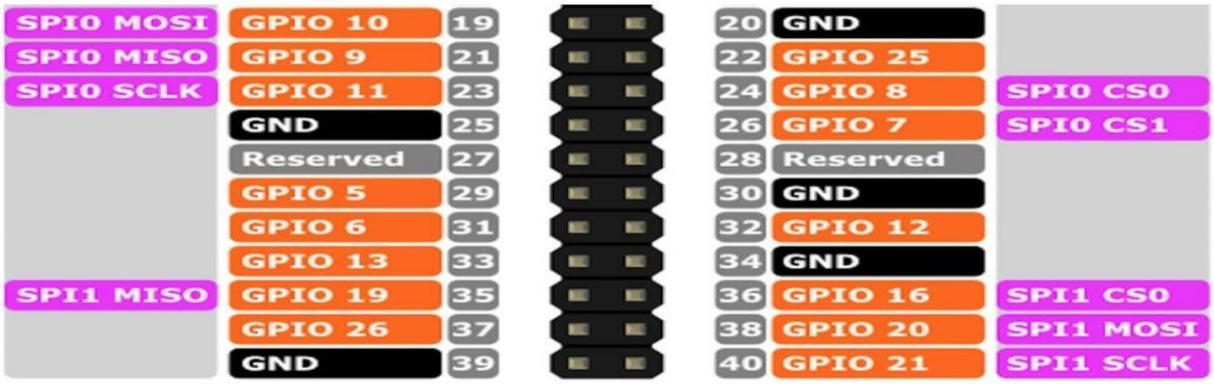
Note that the SDA and SCL pins on the Raspberry Pi are alternate functions for GPIO 2 and 3. When you use a library (Python, Cpp, etc) for I2C, those two GPIOs will be configured so they can use their alternate function.

Some of the best and easy-to-use libraries for I2C are SMBus for Python and WiringPi for Cpp.

SPI

Serial Peripheral Interface (SPI) is a synchronous serial data protocol used for communicating with one or more peripheral devices.

SPI is yet another hardware communication protocol. It is a master-slave bus protocol. It requires more wires than I2C, but can be configured to run faster.



As you can see, you get 2 SPIs by default: SPI0 and SPI1. It means you can use the Raspberry Pi as a SPI master on two different SPI buses at the same time.

And, as for I2C, SPI uses the alternate functions of GPIOs.

For using SPI you'll need 5 pins:

- GND: what a surprise! Make sure you connect all GND from all your slave components and the Raspberry Pi together.
- SCLK: clock of the SPI. Connect all SCLK pins together.
- MOSI: means Master Out Slave In. This is the pin to send data from the master to a slave.
- MISO: means Master In Slave Out. This is the pin to receive data from a slave to the master.
- CS: means Chip Select. Pay attention here: you'll need one CS per slave on your circuit. By default you have two CS pins (CS0 – GPIO 8 and CS1 – GPIO 7). You can configure more CS pins from the other available GPIOs.

In your code, you can use the spidev library for Python, and WiringPi for Cpp.

Raspberry Pi Serial Communication standards

Serial Transfer Schemes

Serial communication uses two methods

- Synchronous.
- Asynchronous.

Synchronous:

- Transfers a block of data (packets or frame) at a time.
- Requires clock signal
- Synchronous transmission is fast.
- Synchronous transmission is costly.
- In Synchronous transmission, time interval of transmission is constant.
- In Synchronous transmission, there is no gap present between data.
- The data blocks are grouped and spaced in regular intervals and are preceded by special characters called syn or synchronous idle characters. See the following illustration.

Figure 1. Synchronous transmission frame format

- After the sync characters are received by the remote device, they are decoded and used to synchronize the connection. After the connection is correctly synchronized, data transmission may begin.

Example: 1. SPI (serial peripheral interface),
2. I2C (inter integrated circuit).
3. Ethernet

Asynchronous:

- Ttransfers single byte at a time
- No need of clock signal.
- Asynchronous transmission is slow.
- Asynchronous transmission economical.
- In asynchronous transmission, time interval of transmission is not constant, it is random.
- In asynchronous transmission, There is present gap between data.

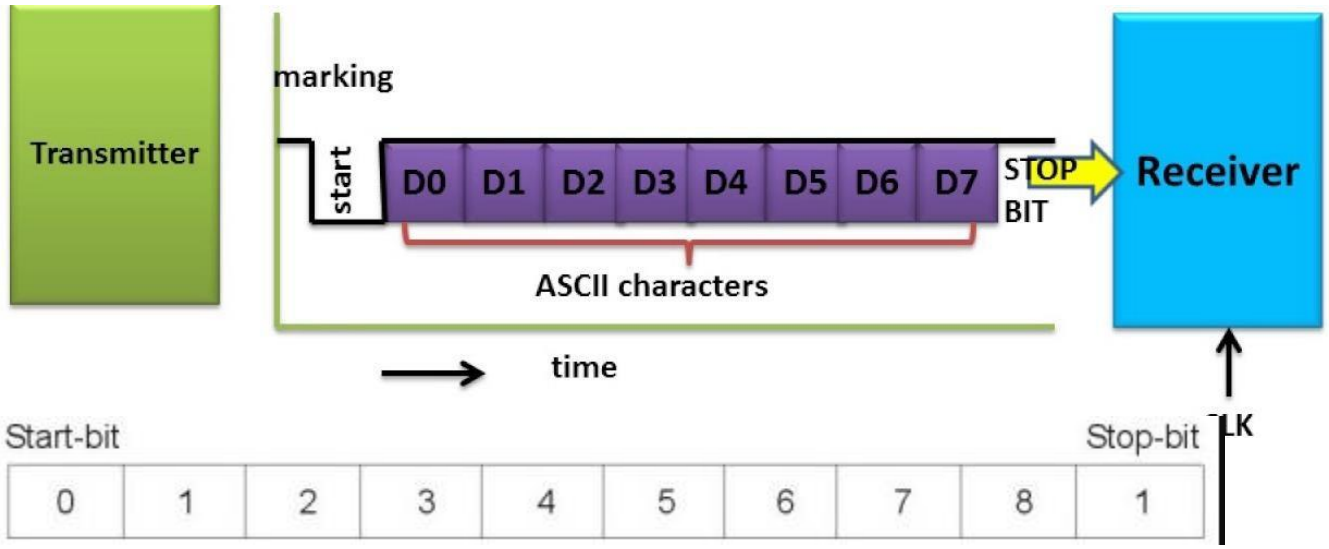


Figure 1.ASynchronous transmission frame format

Example: 1.UART (universal asynchronous receiver transmitter)

2. USB

3. CAN

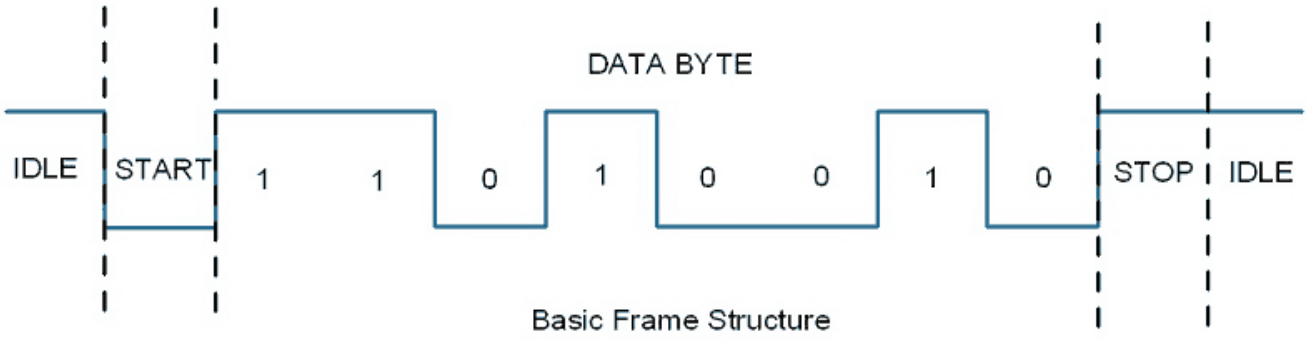
Raspberry Pi UART Communication

Introduction

UART (Universal Asynchronous Receiver/Transmitter) is a serial communication protocol in which data is transferred serially i.e. bit by bit. Asynchronous serial communication is widely used for byte oriented transmission. In Asynchronous serial communication, a byte of data is transferred at a time.

UART serial communication protocol uses a defined frame structure for their data bytes. Frame structure in Asynchronous communication consists:

- **START bit:** It is a bit with which indicates that serial communication has started and it is always low.
- **Data bits packet**: Data bits can be packets of 5 to 9 bits. Normally we use 8-bit data packet, which is always sent after the START bit.
- **STOP bit**: This usually is one or two bits in length. It is sent after data bits packet to indicate the end of frame. Stop bit is always logic high.



Basic Frame Structure

Usually, an asynchronous serial communication frame consists of a START bit (1 bit) followed by a data byte (8 bits) and then a STOP bit (1 bit), which forms a 10-bit frame as shown in the figure above. The frame can also consist of 2 STOP bits instead of a single bit, and there can also be a PARITY bit after the STOP bit.

Raspberry Pi UART

Raspberry Pi has two in-built UART which are as follows:

- **PL011 UART**
- **mini UART**

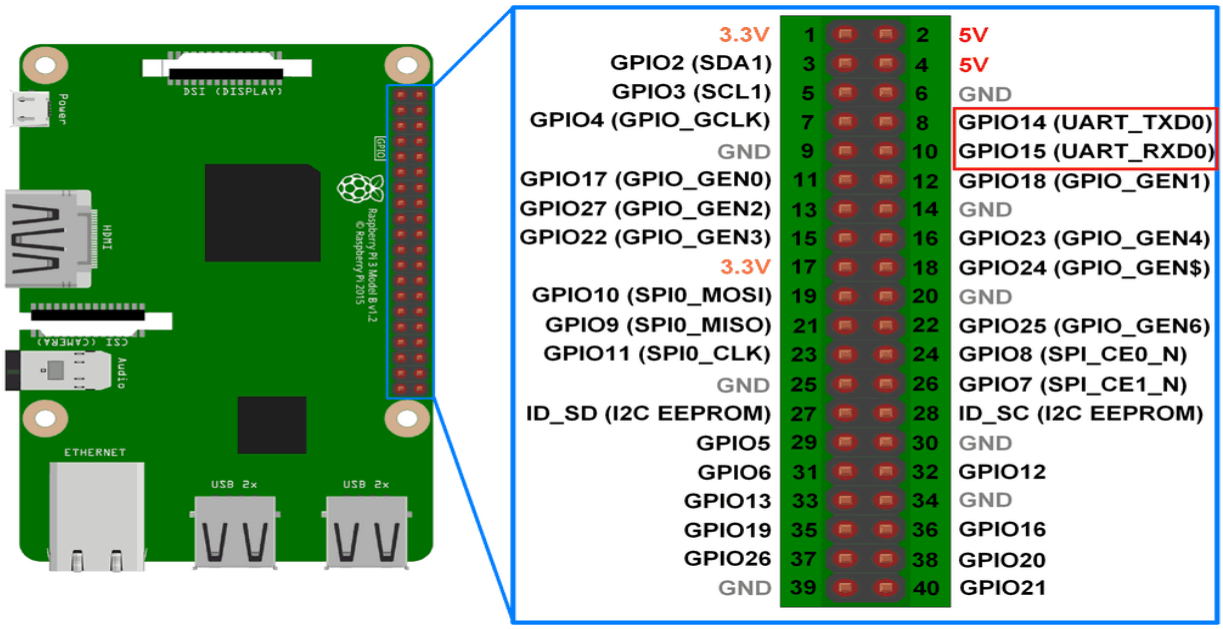
PL011 UART is an ARM based UART. This UART has better throughput than **mini** UART.

In Raspberry Pi 3, mini UART is used for Linux console output whereas PL011 is connected to the On-board Bluetooth module.

And in the other versions of Raspberry Pi, PL011 is used for Linux console output.

Mini UART uses the frequency which is linked to the core frequency of GPU. So as the GPU core frequency changes, the frequency of UART will also change which in turn will change the baud rate for UART. This makes the mini UART unstable which may lead to data loss or corruption. To make mini UART stable, fix the core frequency. mini UART doesn't have parity support.

The PL011 is a stable and high performance UART. For better and effective communication use PL011 UART instead of mini UART.

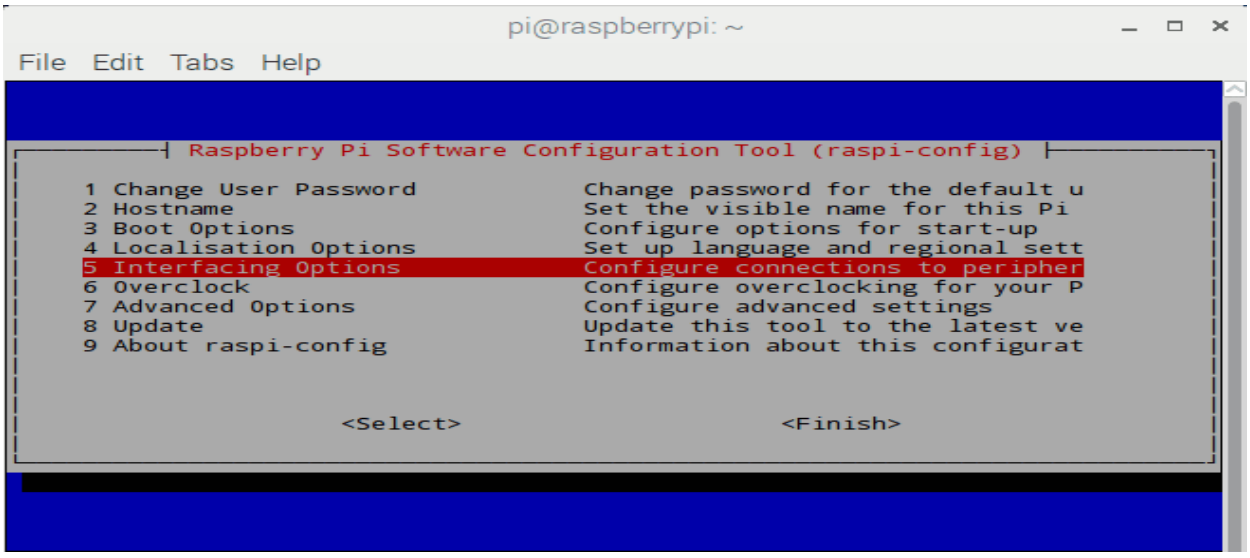


Configure UART on Raspberry Pi

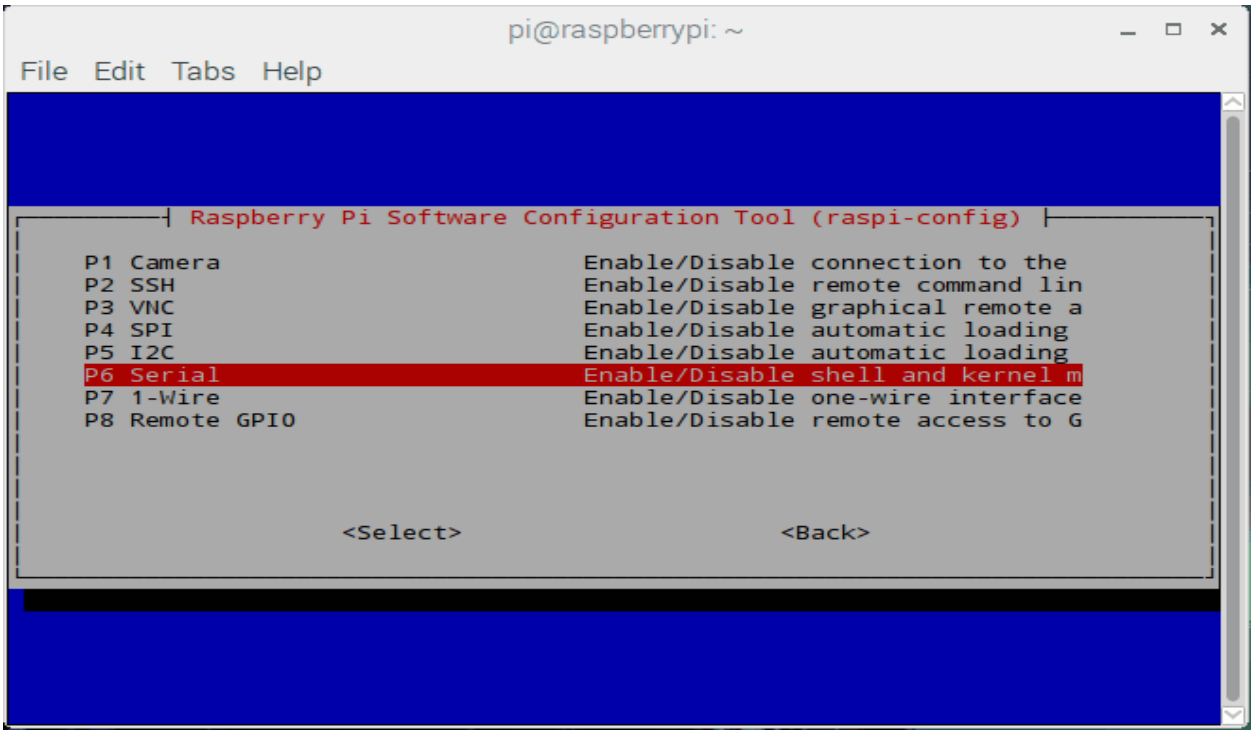
In Raspberry Pi, enter following command in Terminal window to enable UART,

```
sudo raspi-config
```

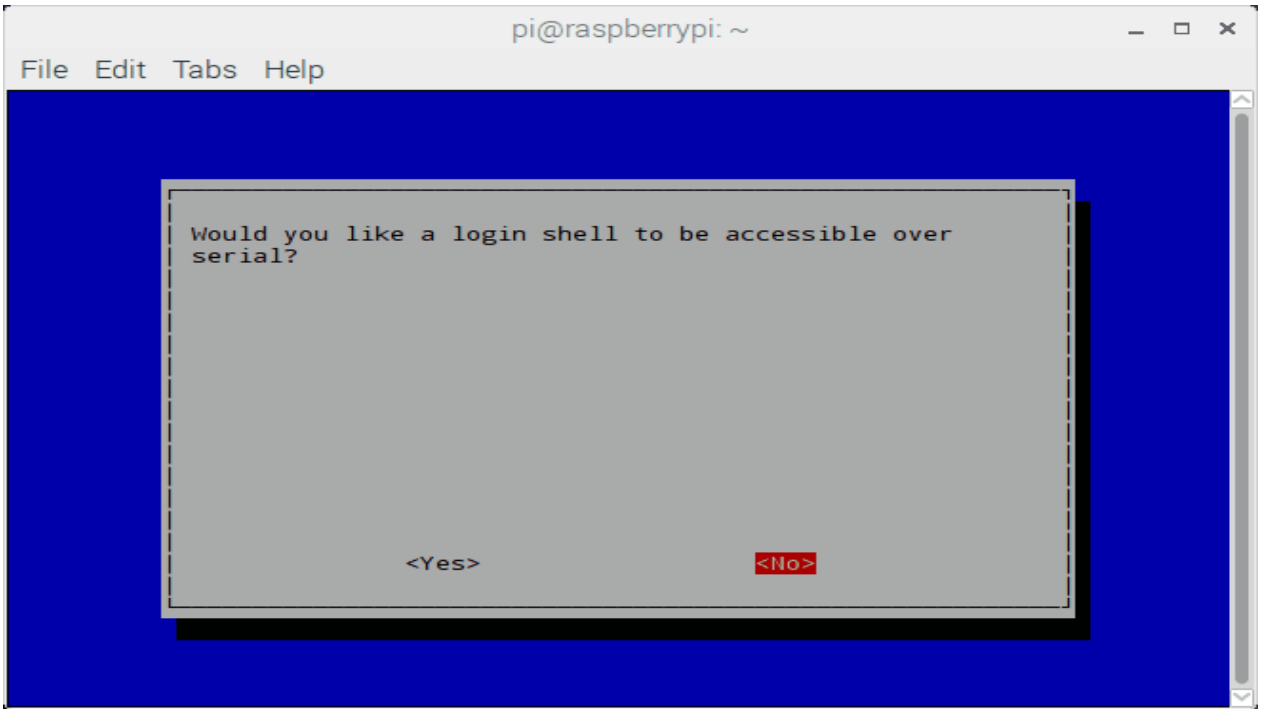
Select -> **Interfacing Options**



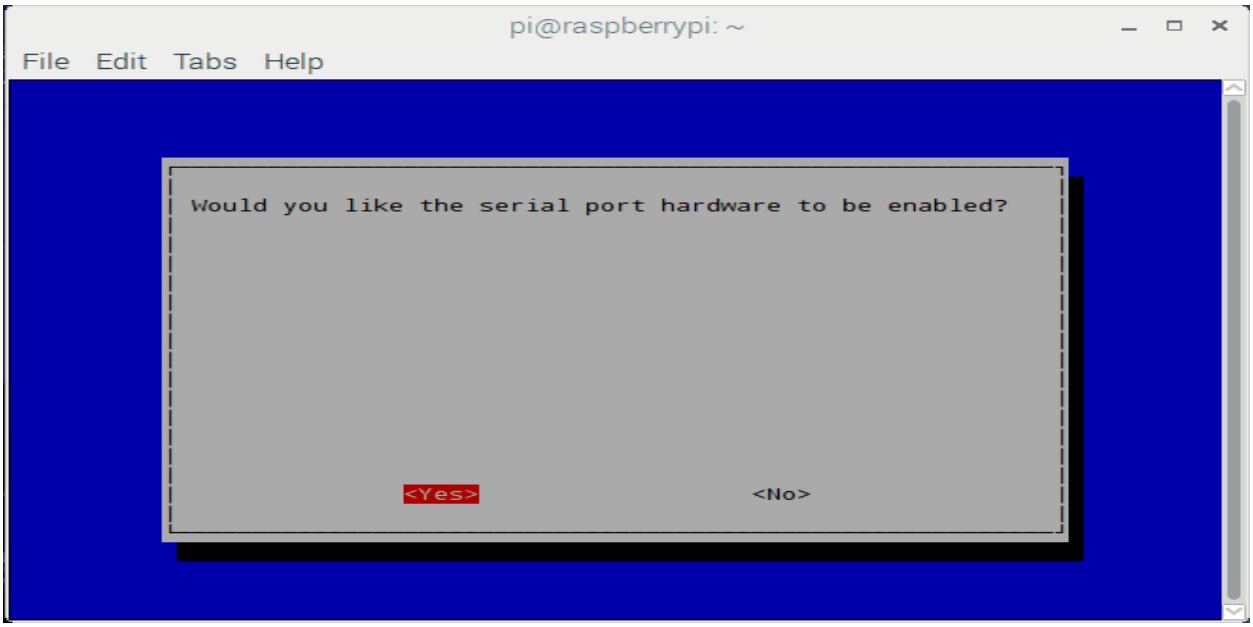
After selecting Interfacing option, select **Serial** option to enable UART



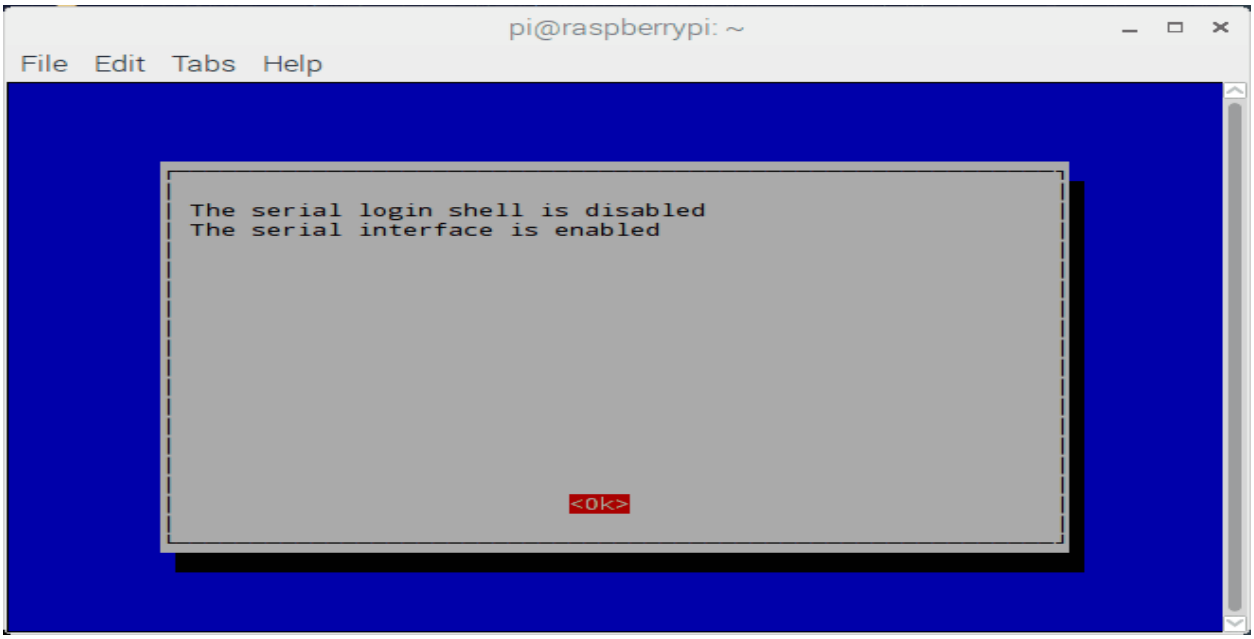
Then it will ask for login shell to be accessible over Serial, select **No** shown as follows.



At the end, it will ask for enabling Hardware Serial port, select **Yes**,



Finally, our UART is enabled for Serial Communication on RX and TX pin of Raspberry Pi 3.



I2C:

Introduction

- I2C (Inter Integrated Circuit) is a synchronous serial protocol that communicates data between two devices.
- It is a master-slave protocol which may have one master or many master and many slaves whereas SPI has only one master.
- It is generally used for communication over short distance.
- The I2C device has 7-bit or 10-bit unique address. So, to access these devices, master should address them by the 7-bit or 10-bit unique address.

- I2C is used in many applications like reading RTC (Real time clock), accessing external EEPROM memory. It is also used in sensor modules like gyro, magnetometer etc.
- It is also called as Two Wire Interface (TWI) protocol.

Raspberry Pi I2C

- Raspberry Pi has Broadcom processor having Broadcom Serial Controller (BSC) which is a master, fast-mode (400Kb/s) BSC controller. The BSC bus is compliant with the Philips I2C bus.
- It supports both 7-bit and 10-bit addressing.
- It also has BSC2 master which is dedicatedly used with HDMI interface and should not be accessed by user.
- I2C bus/interface is used to communicate with the external devices like RTC, MPU6050, Magnetometer, etc with only 2 lines. We can connect more devices using I2C interface if their addresses are different.

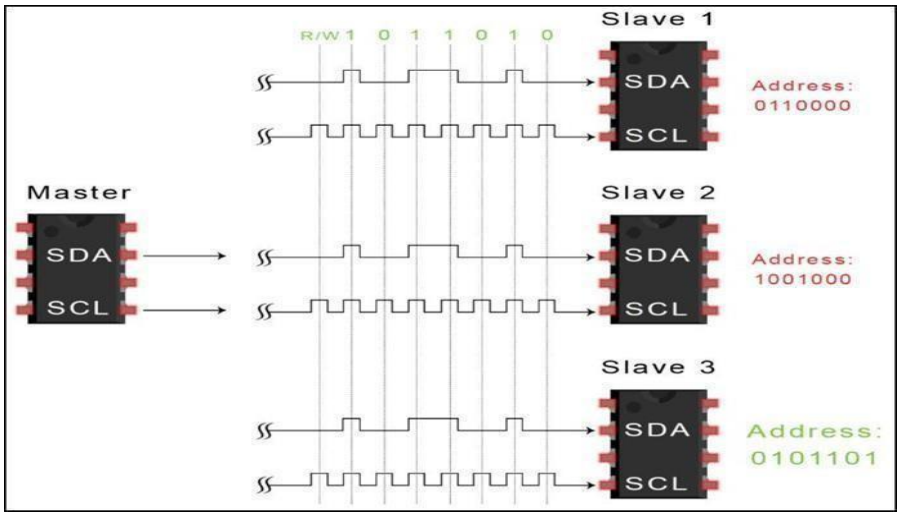
STEPS OF I2C DATA TRANSMISSION

1. The master sends the start condition to every connected slave by switching the SDA line from a high voltage level to a low voltage level *before* switching the SCL line from high to low:

2. The master sends each slave the 7 or 10 bit address of the slave it wants to communicate with, along with the read/write bit:

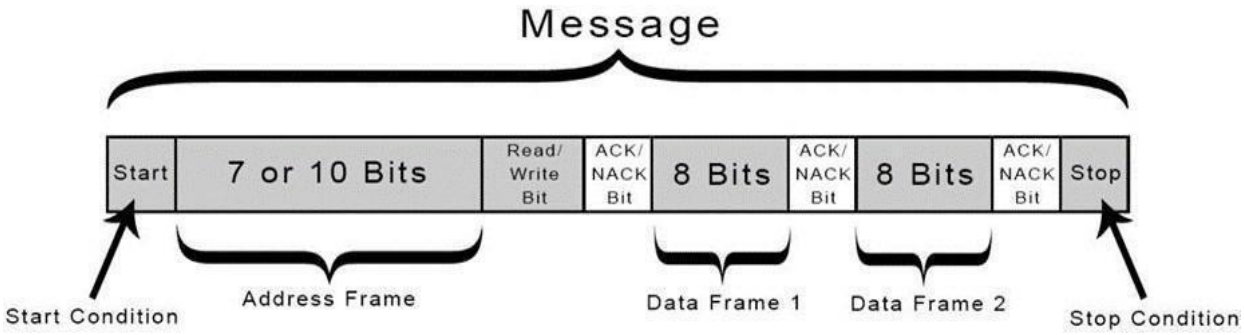
3. Each slave compares the address sent from the master to its own address. If the address matches, the slave returns an ACK bit by pulling the SDA line low for one bit. If the address from the master does not match the slave's own address, SDA line high.

4. The master sends or receives the data frame:



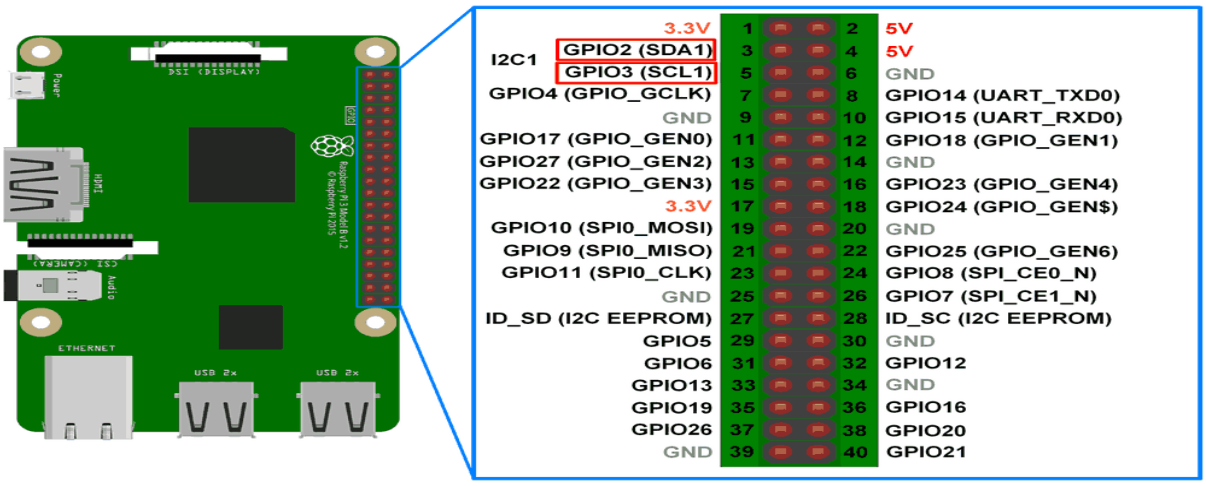
5. After each data frame has been transferred, the receiving device returns another ACK bit to the sender to acknowledge successful receipt of the frame:

6. To stop the data transmission, the master sends a stop condition to the slave by switching SCL high before switching SDA high:



to access I2C bus in Raspberry Pi, we should make some extra configuration. Raspberry Pi has I2C pins which are given as follows.

Raspberry Pi I2C Pins

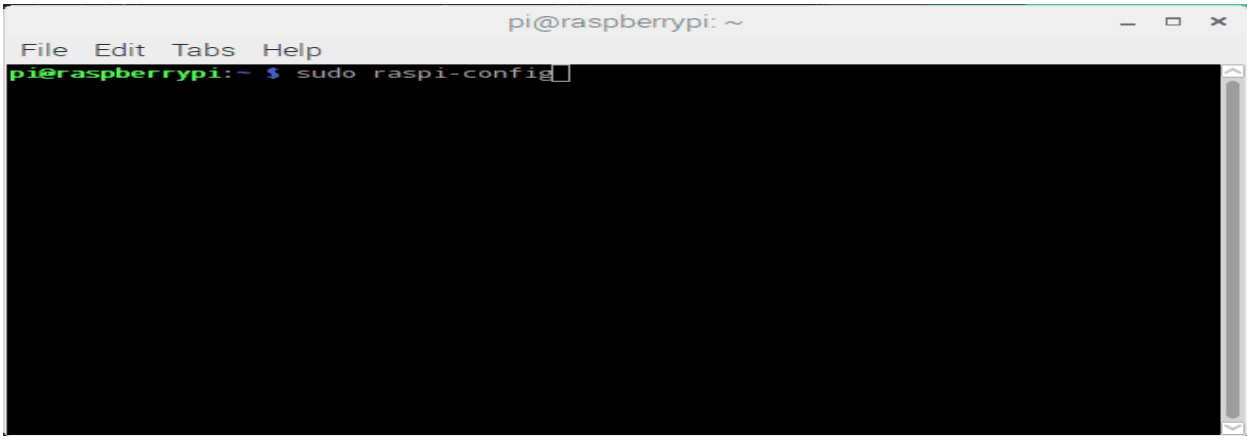


Raspberry Pi I2C Configurations

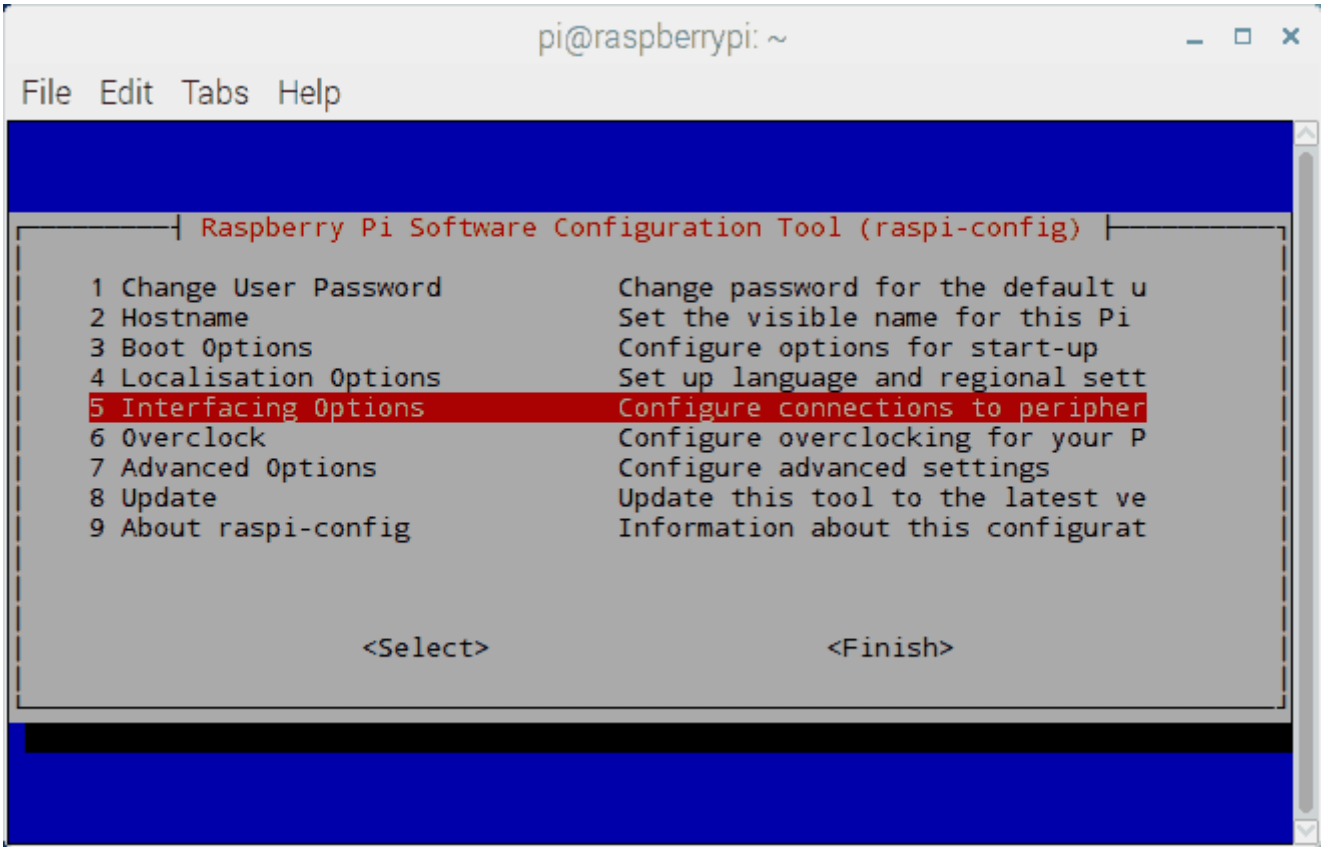
Before start interfacing I2C devices with Raspberry some prior configurations need to be done. These configurations are given as follows:

First, we should enable I2C in Raspberry Pi. We can enable it through terminal which is given below:

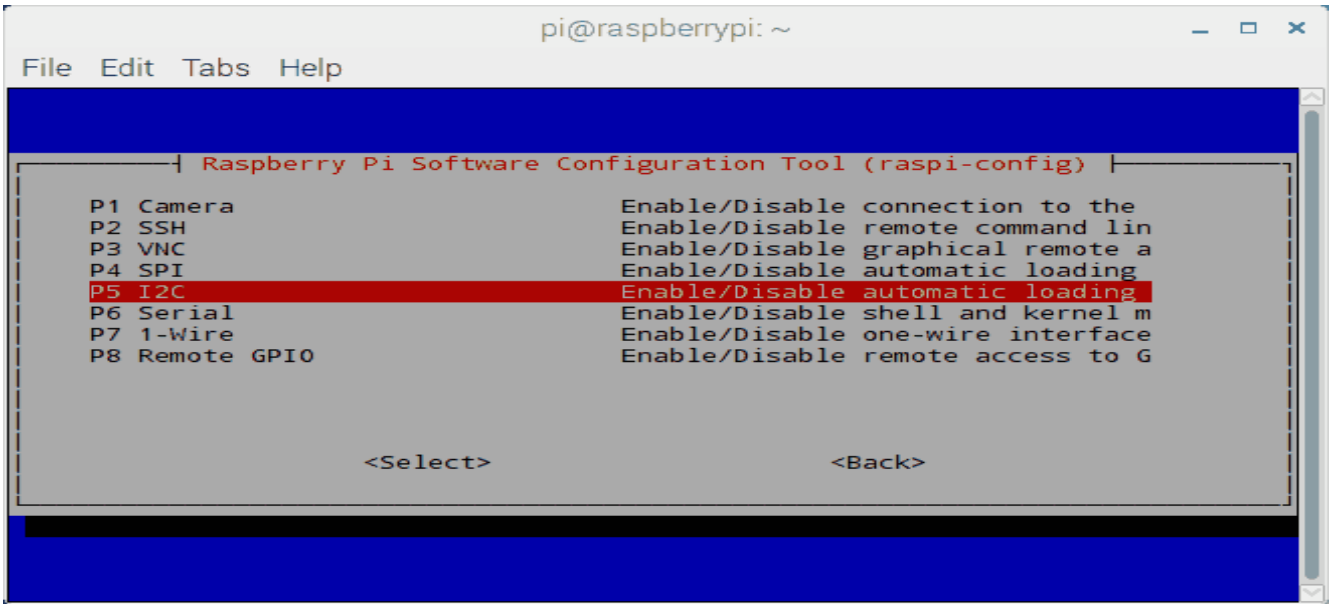
```
sudo raspi-config
```



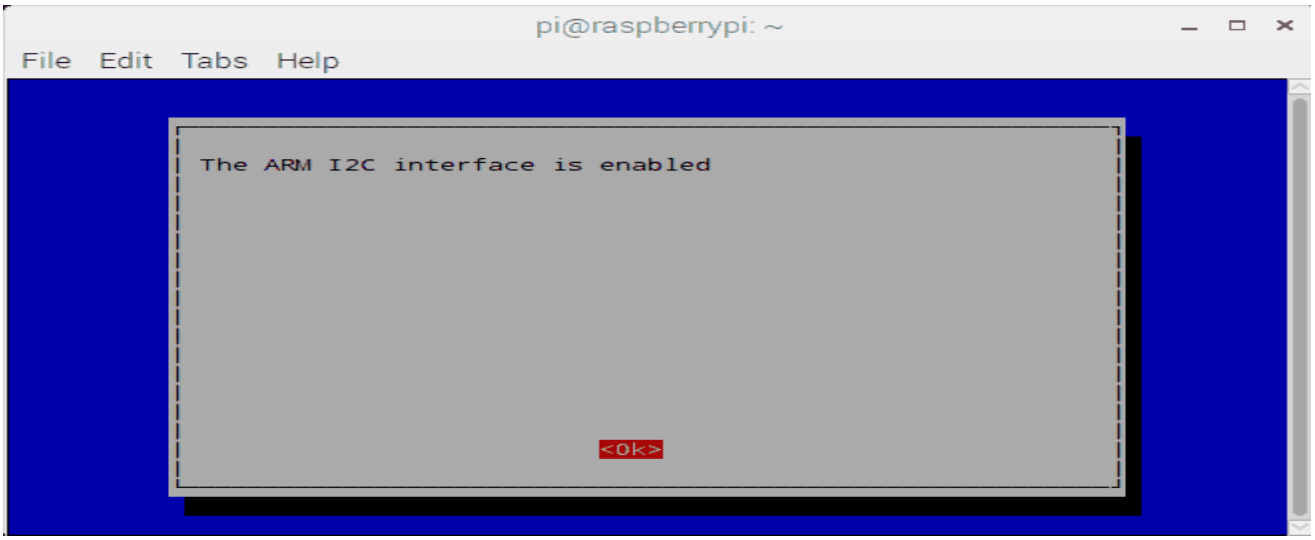
Select **Interfacing Configurations**



- In Interfacing option, Select -> **I2C**



- **Enable I2C configuration**

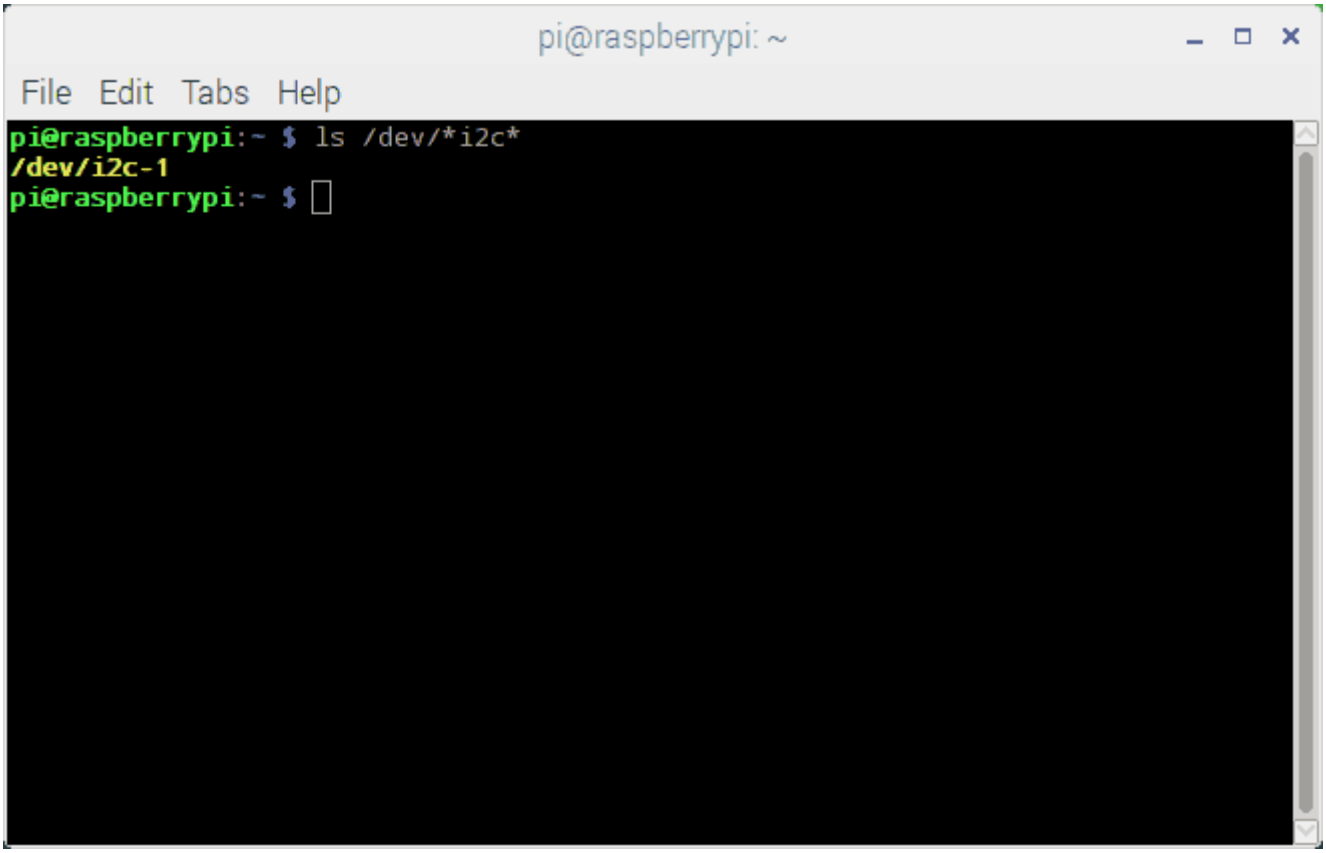


Select **Yes** when it asks to **Reboot.**

Now, after booting raspberry Pi, we can check user-mode I2C port by entering following command.

```
ls /dev/*i2c*
```

then Pi will respond with name of i2c port.



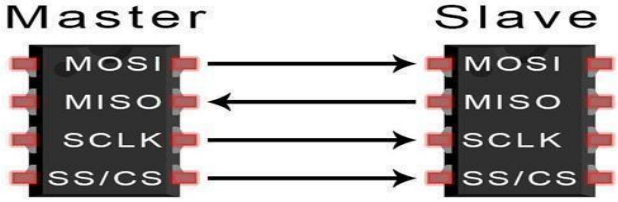
Above response represents the user-mode of I2C interface. Older versions of Raspberry pi may respond with **i2c-0** user-mode port.

SPI COMMUNICATION PROTOCOL

SPI is a common communication protocol used by many different devices. For example, SD card modules, RFID card reader modules, and 2.4 GHz wireless transmitter/receivers all use SPI to communicate with microcontrollers.

One unique benefit of SPI is the fact that data can be transferred without interruption. Any number of bits can be sent or received in a continuous stream.

Devices communicating via SPI are in a master-slave relationship. The master is the controlling device (usually a microcontroller), while the slave (usually a sensor, display, or memory chip) takes instruction from the master. The simplest configuration of SPI is a single master, single slave system, but one master can control more than one slave (more on this below).



MOSI (Master Output/Slave Input) – Line for the master to send data to the slave.

MISO (Master Input/Slave Output) – Line for the slave to send data to the master **SCLK (Clock)** –

Line for the clock signal.

SS/CS (Slave Select/Chip Select) – Line for the master to select which slave to send data to.

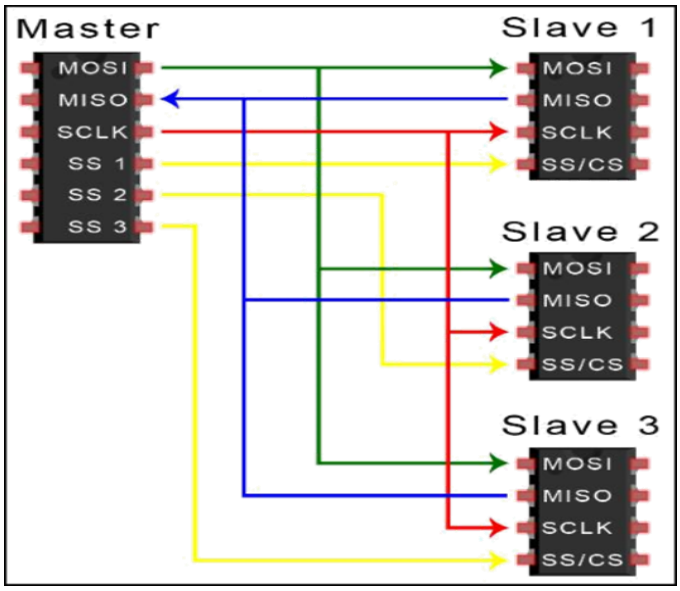
CLOCK

- The clock signal synchronizes the output of data bits from the master to the sampling of bits by the slave. One bit of data is transferred in each clock cycle, so the speed of data transfer is determined by the frequency of the clock signal. SPI communication is always initiated by the master since the master configures and generates the clock signal.
- Any communication protocol where devices share a clock signal is known as *synchronous*. SPI is a synchronous communication protocol.

SLAVE SELECT

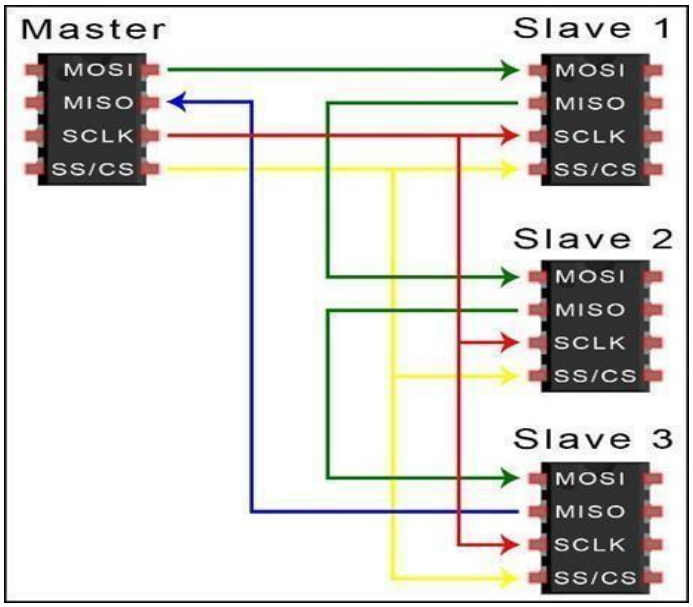
The master can choose which slave it wants to talk to by setting a low voltage level. In the idle, non-transmitting state, the slave select line is kept at a high voltage level. Multiple CS/SS pins may be available on the master, which allows for multiple slaves to be wired in parallel.

MULTIPLE SLAVES



SPI can be set up to operate with a single master and a single slave, and it can be set up with multiple slaves controlled by a single master. There are two ways to connect multiple slaves to the master. If the master has multiple slave select pins, the slaves can be wired in parallel like this:

If only one slave select pin is available, the slaves can be daisy-chained like this:

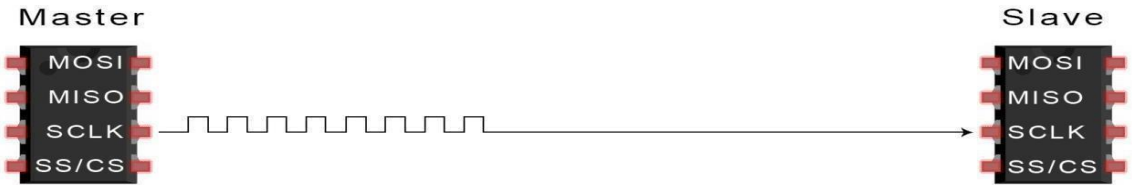


MOSI AND MISO

The master sends data to the slave bit by bit, in serial through the MOSI line. The slave receives the data sent from the master at the MOSI pin. Data sent from the master to the slave is usually sent with the most significant bit first.The slave can also send data back to the master through the MISO line in serial. The data sent from the slave back to the master is usually sent with the least significant bit first.

STEPS OF SPI DATA TRANSMISSION

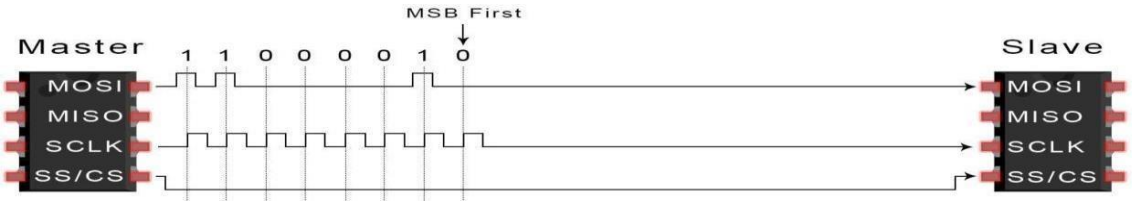
1. The master outputs the clock signal:



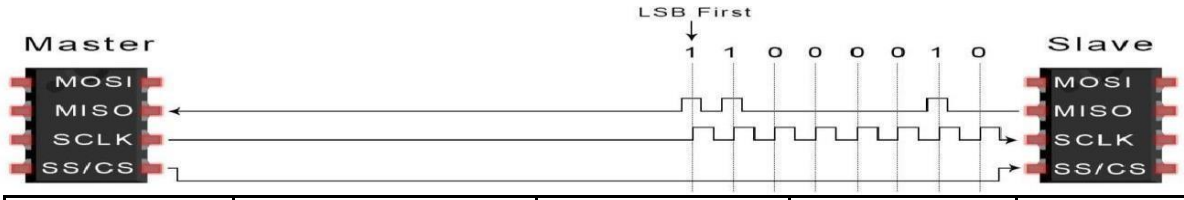
2. The master switches the SS/CS pin to a low voltage state, which activates the slave:

3. The master sends the data one bit at a time to the slave along the MOSI line. The slave reads the

bits as they are received:



4. If a response is needed, the slave returns data one bit at a time to the master along the MISO line. The master reads the bits as they are received:



Serial Protocol	**Synchronous /Asynchronous**	**Type**	**Duplex**	**Data transfer rate (kbps)**
UART	Asynchronous	peer-to-peer	Full-duplex	20
I2C	Synchronous	master/slave multi-master	Half-duplex	3400
SPI	Synchronous	master/slave multi-master	Full-duplex	>1,000

Raspberry Pi Programming Basics

1. Python RPi.GPIO API Library

- **One of the** famous library in launching input and output pins is the RPi.GPIO library. You can use this library in python programming.

- Use the RPi.GPIO module as the driving force behind our Python GPIO examples. This set of Python files and source is included with Raspbian OS --NOOBS,(Linux based)

2. How to Setup RPi.GPIO

- This library is available on Raspbian operating system by default and you don't need to install it.

- In order To us RPi.GPIO in Python script, you need to put this statement at the top of your file: **import RPi.GPIO as GPIO**

- That statement "includes" the RPi.GPIO module, and goes a step further by providing a local name -- GPIO -- which we'll call to reference the module from here on.

3. **Pin Numbering Declaration**

- After including the RPi.GPIO module, the next step is to determine which of the two **pin-numbering schemes**

- 1. **GPIO.BOARD** : Board numbering scheme. The pin numbers follow the pin numbers on header P1.

- 2. **GPIO.BCM :** Broadcom chip-specific pin numbers. These pin numbers follow the lower-level numbering system defined by the Raspberry Pi's Broadcom-chip brain.

- To specify in your code which number-system is being used, use the **GPIO.setmode() function.**

Example:

GPIO.setmode(GPIO.BCM)# *will activate the Broadcom-chip specific pin numbers.*

Note: Both the import and setmode lines of code are required, if you want to use Python.

4. **Setting a Pin Mode:**

- If you've used Arduino, you're probably familiar with declare a "pin mode" before you can use it as either an input or output.

- In Raspberry Pi to set a pin mode, use the

GPIO.setup(pin number, pinmode) function.

- **Example:** if you want to set pin 18 as an output, write:

- **GPIO.setup(18, GPIO.OUT)**

Note: Remember that the pin number will change if you're using the board numbering system (instead of 18, it'd be 12).

GPIO.setmode(GPIO.BCM); Using pins BCM numbers

Basic Commands

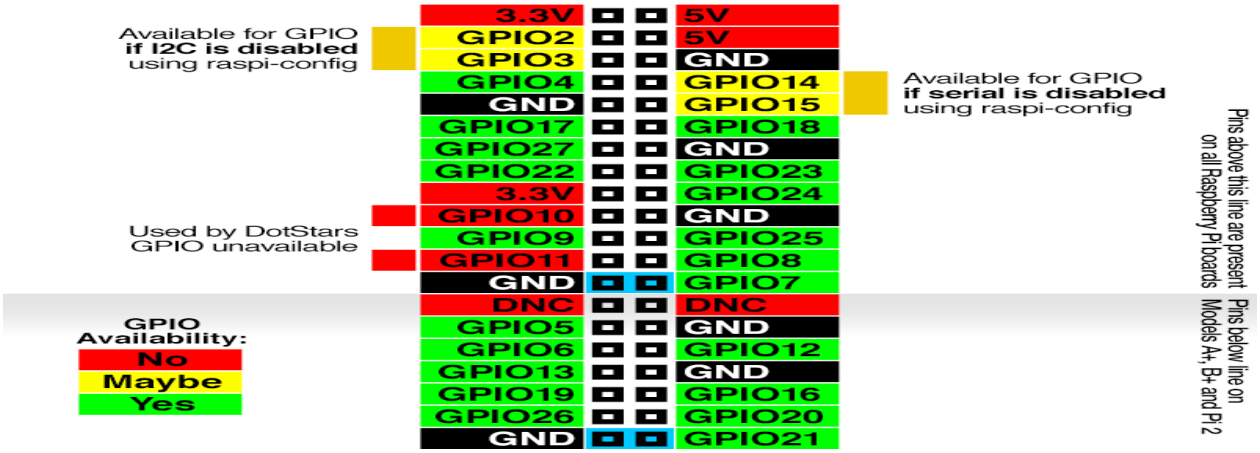
- GPIO.setup(pin, GPIO.IN); Determines the pin as input

- GPIO.setup(pin, GPIO.OUT); Determines the pin as an output

- GPIO.input(pin): Reading input pin

- GPIO.output(pin, state): Writing on the output pin.

Broadcom (BCM) pin names



Interfacing Raspberry Pi with Basic Peripherals

1. Interfacing LED with Raspberry Pi

LED:

- The LED is a special type of diode and they have similar electrical characteristics to a PN junction diode. Hence the LED allows the flow of current in the forward direction and blocks the current in the reverse direction.

LED Symbol

- The LED symbol is similar to a diode symbol except for two small arrows that specify the emission of light, thus it is called LED (light-emitting diode). The LED includes two terminals namely anode (+) and the cathode (-). The LED symbol is shown below.

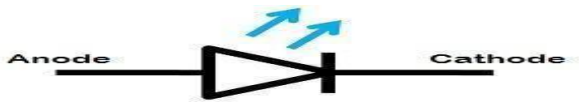


Figure:LED Symbol

Working of LED

- When the voltage is not applied to the LED, then there is no flow of electrons and holes so they are stable. Once the voltage is applied then the LED will forward biased, so the electrons in the N-region and holes from P-region will move to the active region. This region is also known as the depletion region. Because the charge carriers like holes include a positive charge whereas electrons have a negative charge so the light can be generated through the recombination of polarity charges.

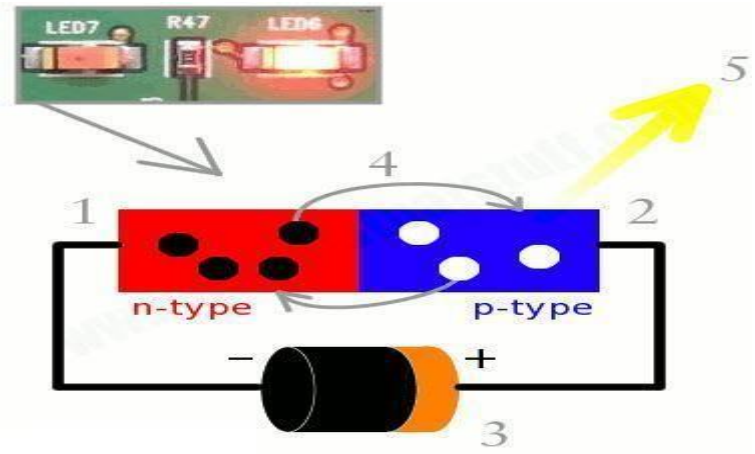


Figure: Working of Light Emitting Diode

2. Resistor:

- A resistor is a passive two-terminal electrical component that implements electrical resistance as a circuit element.
- In electronic circuits, resistors are used to reduce current flow, adjust signal levels, to divide voltages, bias active elements, and terminate transmission lines, among other uses.

Symbol of Resistor

- Resistor is a 2 terminal passive device. The symbol is given below.

Figure: Resistor Symbol

Unit of resistance

- The SI-unit of resistance is Ohm [Ω]. The higher multiple and sub-multiple values of ohm is kilo ohms [KΩ], mega ohms [MΩ], milli ohm and so on.

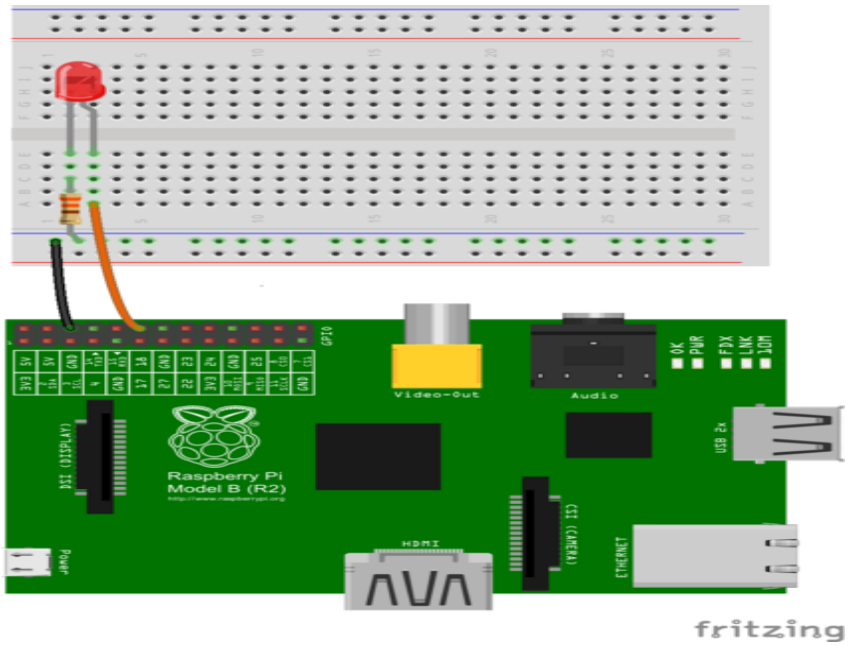


Figure: Interfacing LED with Raspberry Pi Connections

EXAMPLE 1: LED ON/OFF

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setup(18,GPIO.OUT)
while True:
    GPIO.output(18,True)
    time.sleep(2)
    GPIO.output(18,False)
    time.sleep(2)
```

OUTPUT:

Figure: Led OFF

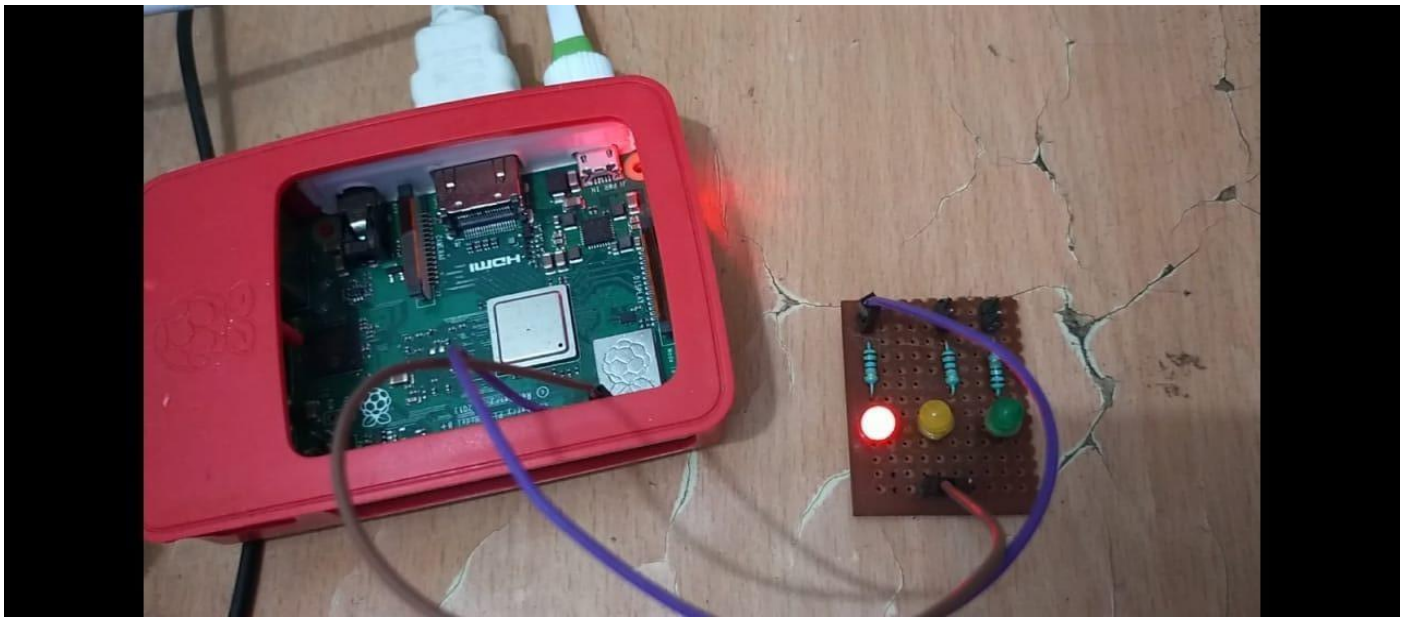
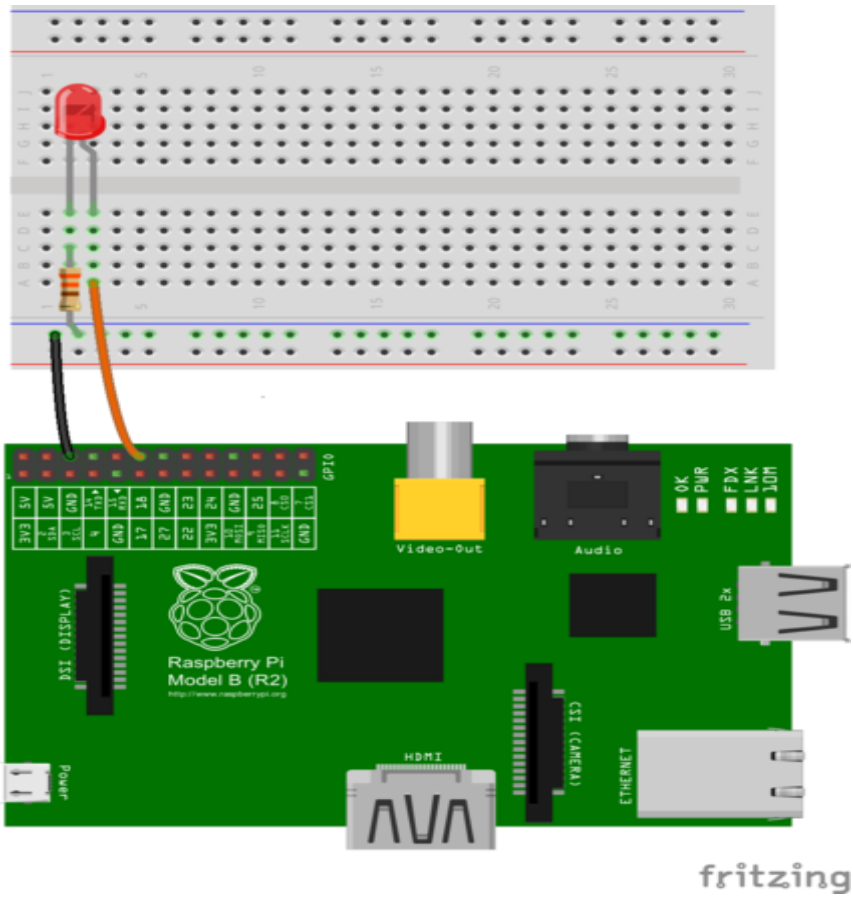


Figure: Led ON

Example 2. LED BLINKING using FOR LOOP




```
import RPi.GPIO as GPIO
import time
 ledPin = 12     # GPIO 17
 delay = 1   # 1s
loopCnt = 10
 def main():
    for i in range(loopCnt):
        GPIO.output(ledPin, GPIO.HIGH)  # output 3.3 V from GPIO pin
        time.sleep(delay)   # delay for 1s
        GPIO.output(ledPin , GPIO.LOW)  # output 0 V from GPIO pin
        time.sleep(delay)   # delay for 1s
 def setup():
    GPIO.setmode(GPIO.BOARD)
    GPIO.setup(ledPin, GPIO.OUT)     # initialize GPIO pin as OUTPUT pin
```

```
    GPIO.output(ledPin, GPIO.LOW)


if __name___== '__main__':
    setup()
    main()
    GPIO.cleanup()  # free up the resources used
```

OUTPUT:

Figure: Led OFF

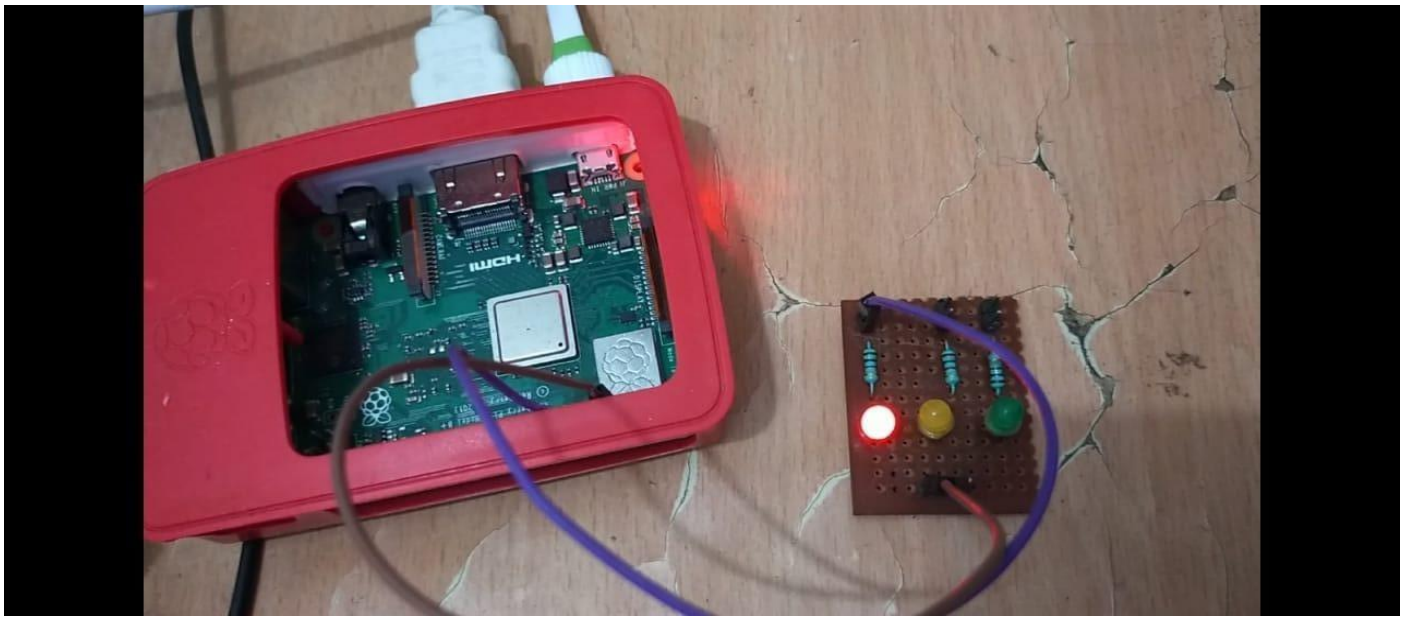


Figure: Led ON

2. Interfacing PUSH BUTTON SWITCH & LED with Raspberry Pi.

Push Button is simplest of devices and it is the basic input device that can be connected to any controller or processor like Arduino or Raspberry Pi.

A push button in its simplest form consists of four terminals. In that, terminals 1 and 2 are internally connected with each other and so are terminals 3 and 4. So, even though you have four terminals, technically, you have only two terminals to use.

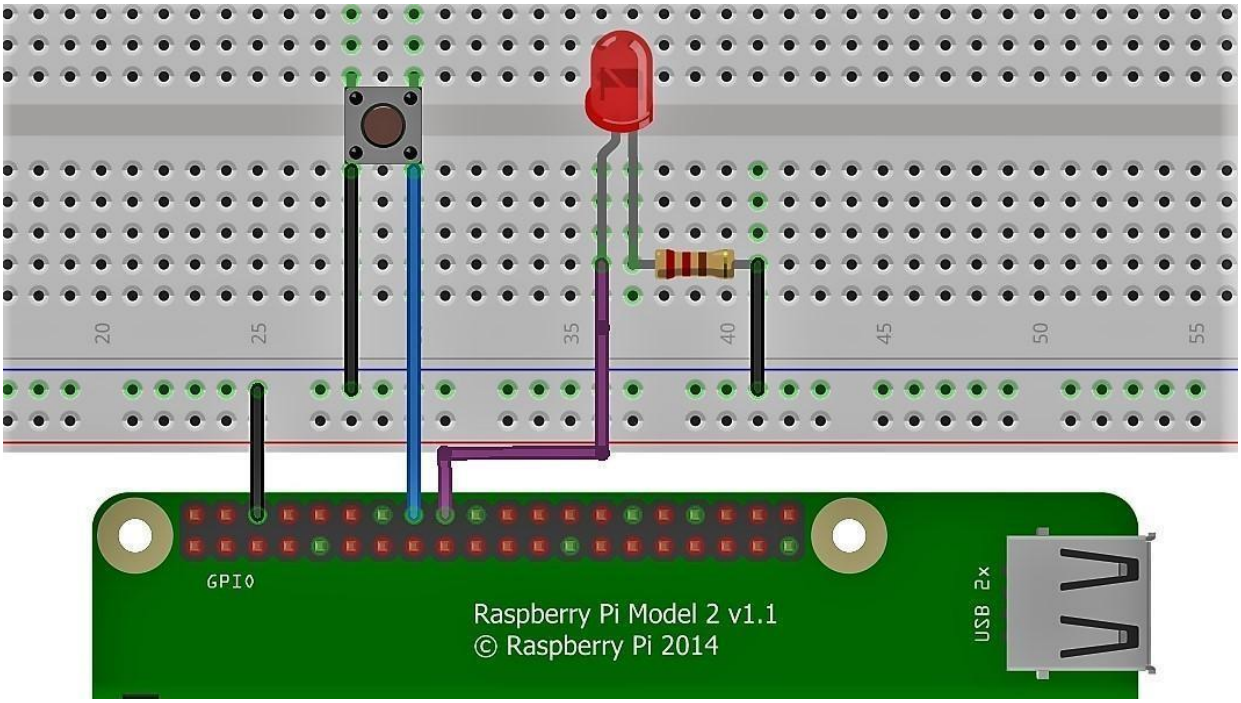


Figure: Interfacing PUSH BUTTON SWITCH & LED with Raspberry Pi.

Example:

```
import RPi.GPIO as GPIO
import time
GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(18, GPIO.OUT)
GPIO.setup(4,GPIO.IN)


# input of the switch will change the state of the LED
while True:
 GPIO.output(18,GPIO.input(4))
 time.sleep(0.05)
```

OUTPUT:

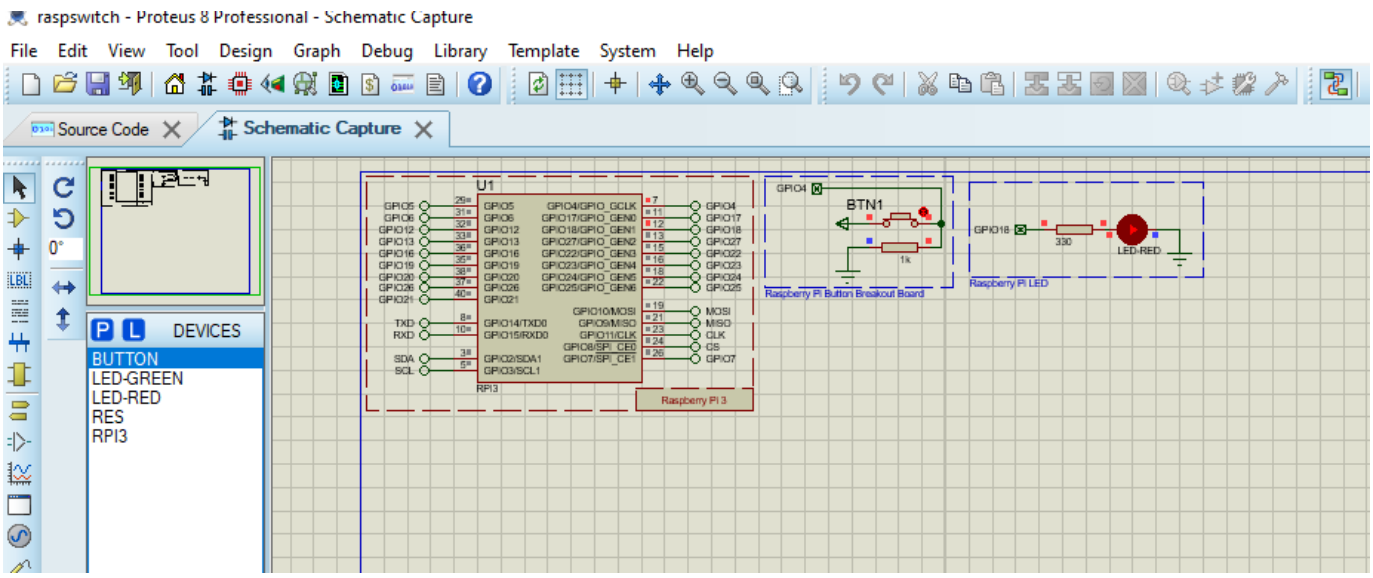


Figure 1: LED ON When Switch is PRESSED

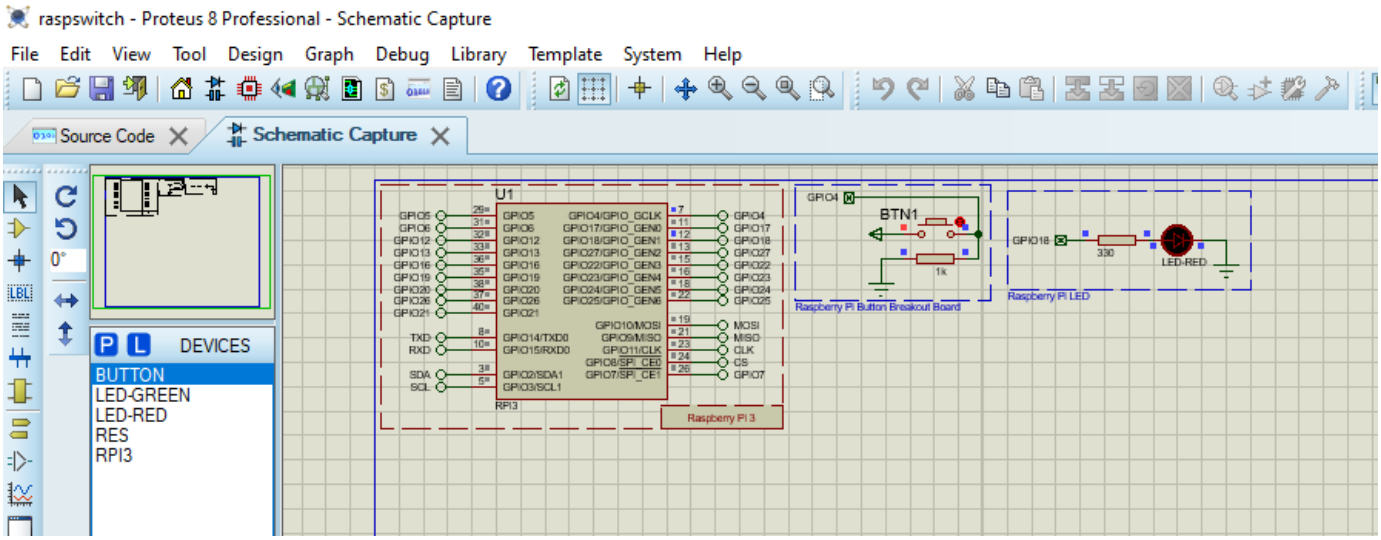
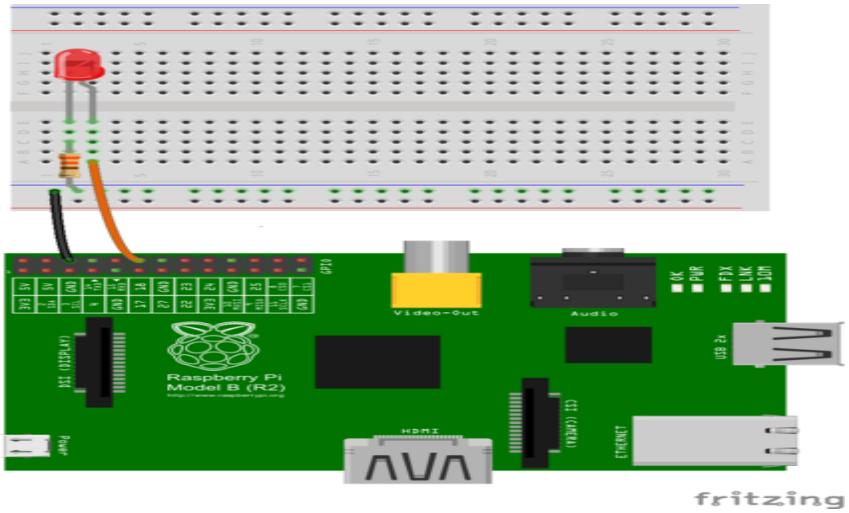


Figure 2: LED OFF When Switch is released

4. PWM Illustration in Raspberry Pi



CODE:

```
import RPi.GPIO as GPIO
from time import sleep
ledpin = 12            # PWM pin connected to LED
GPIO.setwarnings(False)        #disable warnings
GPIO.setmode(GPIO.BOARD)       #set pin numbering system
GPIO.setup(ledpin,GPIO.OUT)
pi_pwm = GPIO.PWM(ledpin,1000)      #create PWM instance with frequency
```

```
pi_pwm.start(0)          #start PWM of required Duty Cycle
while True:
    for duty in range(0,101,1):
        pi_pwm.ChangeDutyCycle(duty) #provide duty cycle in the range 0-100
        sleep(0.01)
    sleep(0.5)

    for duty in range(100,-1,-1):
        pi_pwm.ChangeDutyCycle(duty)
        sleep(0.01)
    sleep(0.5)
```

Figure 1: LED IS OFF(PWM VALUE IS ZERO)

FIGURE 2: LED IS ON WITH WHEN PWM VALUE IS MID RANGE (MEDIUM BRIGHTNESS)

FIGURE 3: LED IS ON WITH WHEN PWM VALUE IS MAXIMUM RANGE(MORE BRIGHTNESS)

3. Interfacing Temperature Sensor with Raspberry Pi.

DHT22:

Temperature: A measure of the warmth or coldness of an object or substance with reference to some standard value**.**

Humidity: A quantity representing the amount of water vapour in the atmosphere.

DHT22: Digital Humidity Temperature sensor

- The DHT-22 (also named as AM2302) is a digital-output relative humidity and temperature sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and spits out a digital signal on the data pin.

Features

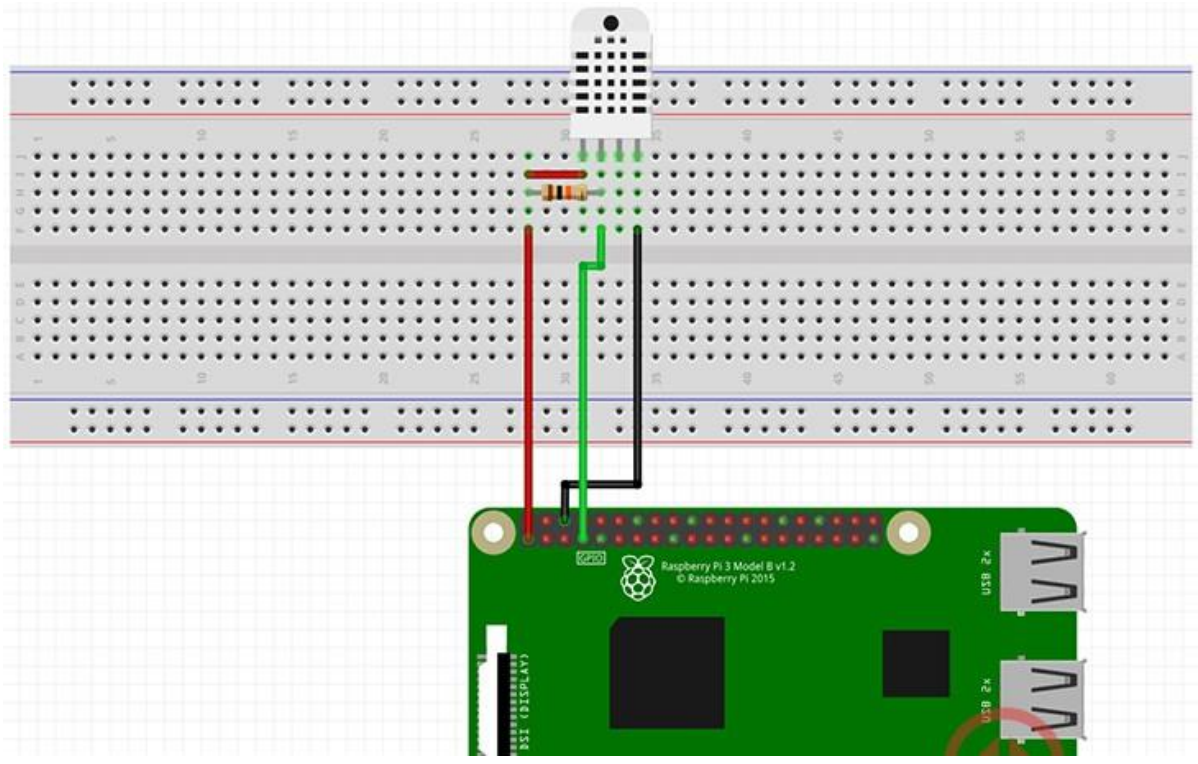
- The range of operating voltage ( ) is 3 V to 5 V power.
- It measures temperature in a range of -40°C to +125°C with an accuracy of ± 0.5 degrees.
- The measuring range for humidity is from 0 to 100% with an accuracy of 2-5%.
- The maximum operating current for DHT22 sensor is 2.5mA.
- The sampling rate for DHT22 sensor is 0.5 Hz. It takes measurement once every 2 seconds.

Pin Description:

The DHT22 sensor is very simple and easy to use. It has only four pins.

1. **Vcc** is the power pin. Apply voltage in a range of 3.5 V to 5.0 V at this pin.
2. **Data Out** is the digital output pin. It sends out the value of measured temperature and humidity in the form of serial data.
3. **N/C** is a not connect pin.
4. **GND:** Connect the GND pin to main ground.

Figure: Pin Configuration of DHT22



PRE REQUISTE library Installations:

1. Before we get started with programming a script for the Raspberry Pi humidity sensor, we must first ensure that we have the latest updates on our Raspberry Pi.

We can do this by running the following two commands to update both the package list and installed packages.

```
Terminal $

  sudo apt-get update
  sudo apt-get upgrade
```

2. Now using pip, we will go ahead and install Adafruit's DHT library to the Raspberry Pi.

We will be using this Python library to interact with our DHT22 Humidity/Temperature sensor.

As a bonus, the library also supports the DHT11 and AM2302 humidity/temperature sensors making it a great library to learn how to utilize.

Run the following command to install the DHT library to your Raspberry Pi.

```
Terminal $

  sudo pip3 install Adafruit_DHT
```

CODE:

```
import Adafruit_DHT

DHT_SENSOR = Adafruit_DHT.DHT22

DHT_PIN = 4

while True:

    humidity, temperature = Adafruit_DHT.read_retry(DHT_SENSOR, DHT_PIN)


    if humidity is not None and temperature is not None:

        print("Temp={0:0.1f}*C  Humidity={1:0.1f}%".format(temperature, humidity))

    else:

        print("Failed to retrieve data from humidity sensor")
```

output

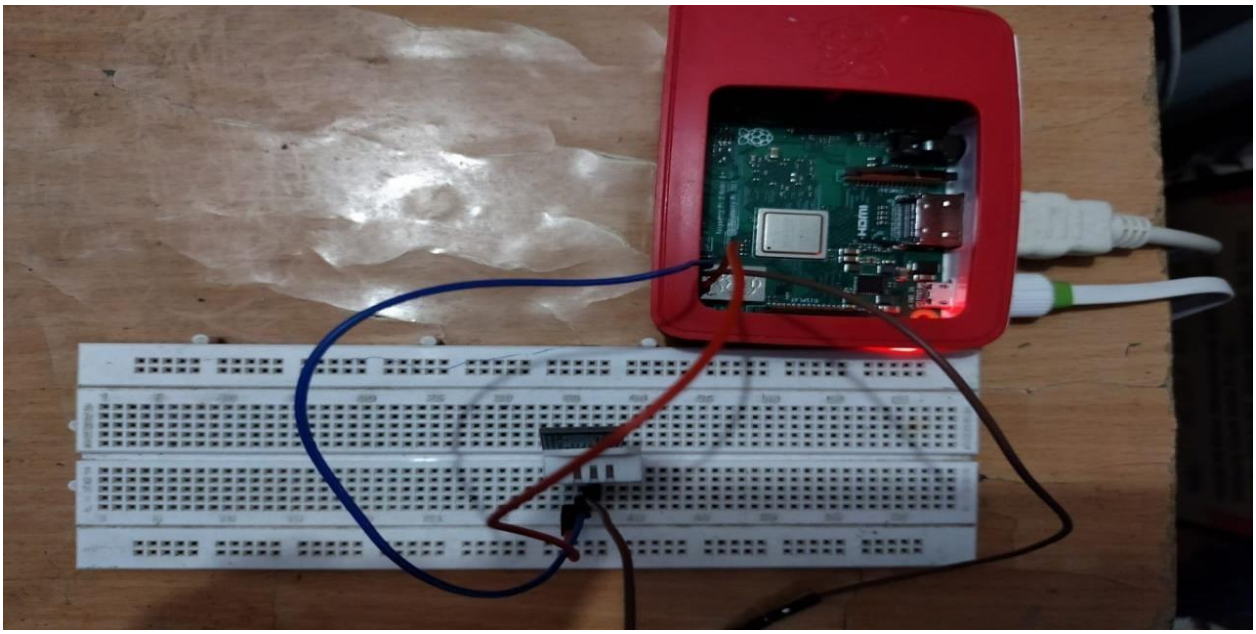


Figure 1: interfacing DHT22 to Raspberry Pi Connections

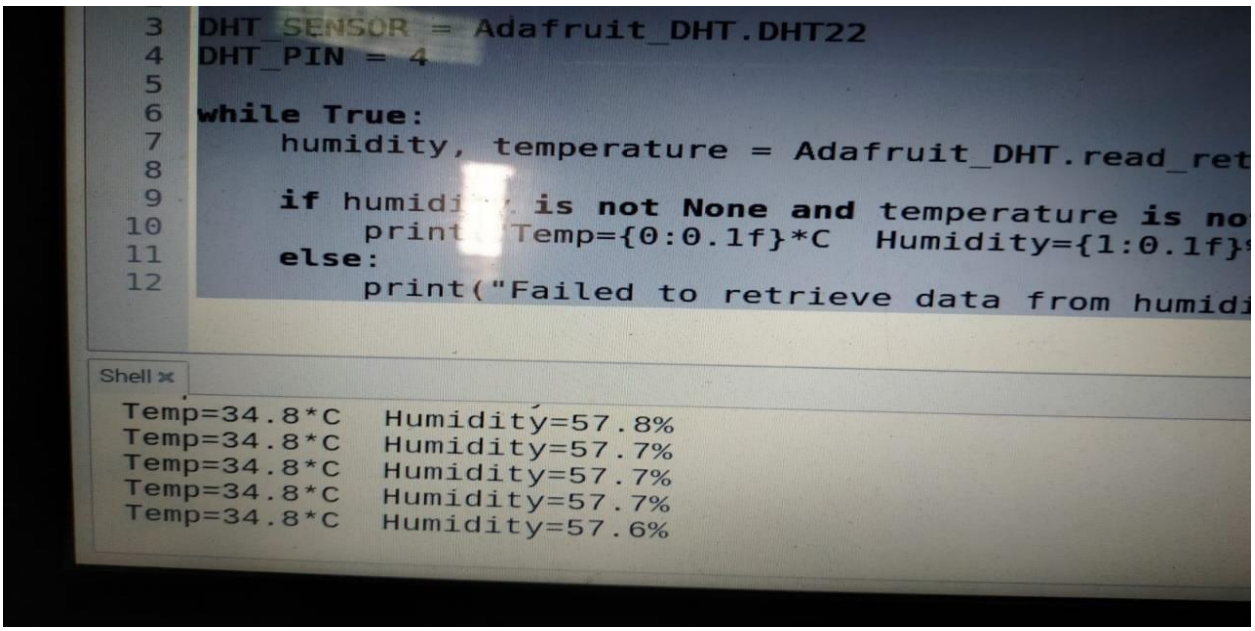


Figure 2: Temperature & Humidity readings at Room Temp

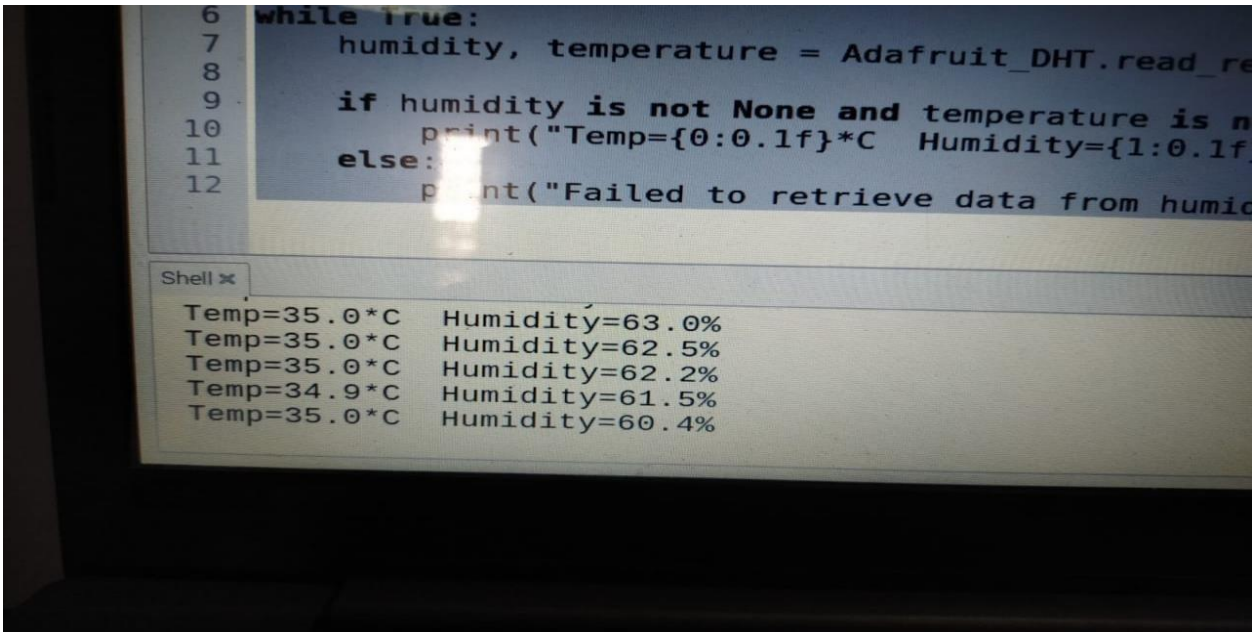


Figure 3: Temperature readings when sensor hold in hand & Humidity readings at Room Temp

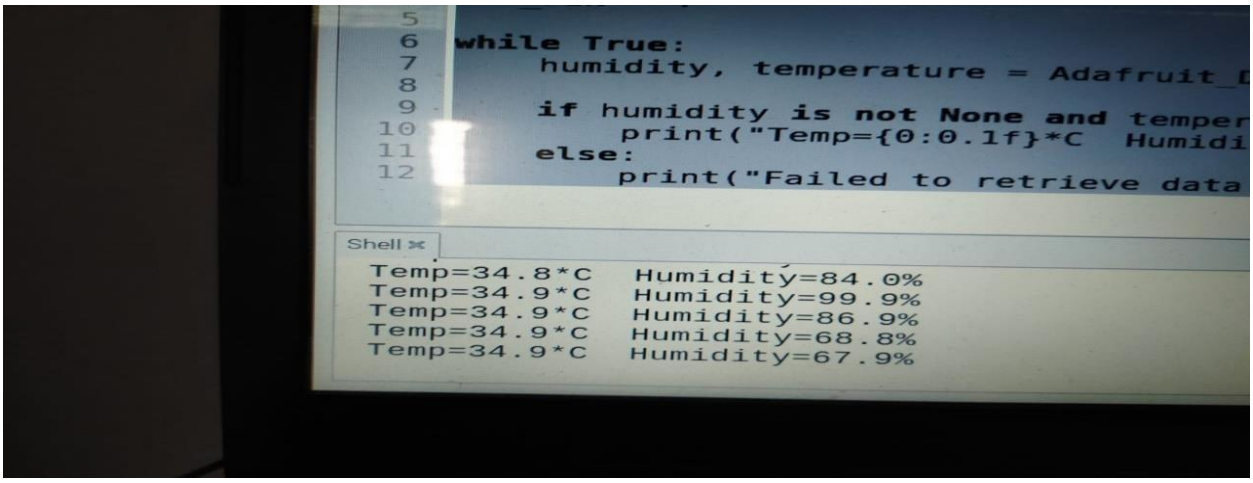


Figure 4: Temperature readings at Room Temperature& Humidity when blown

Interfacing Ultrasonic Sensor with Raspberry Pi

•An ultrasonic sensor is an electronic device that measures the distance of a target object by emitting ultrasonic sound waves, and converts the reflected sound into an electrical signal.

HC-SR04 Ultrasonic Sensor Pin Configuration

- Includes four pins and the pin configuration of this sensor is discussed below.
- Pin1 (Vcc): This pin provides a +5V power supply to the sensor.
- Pin2 (Trigger): This is an input pin, used to initialize measurement by transmitting ultrasonic waves by keeping this pin high for 10us.
- Pin3 (Echo): This is an output pin, which goes high for a specific time period and it will be equivalent to the duration of the time for the wave to return back to the sensor.
- Pin4 (Ground): This is a GND pin used to connect to the GND of the system.

Features:

- The power supply is +5V DC
- Dimension is 45mm x 20mm x 15mm
- Quiescent current used for this sensor is <2mA
- The input pulse width of trigger is10uS
- Operating current is 15mA
- Measuring angle is 30 degrees
- The distance range is 2cm to 800 cm
- Resolution is 0.3 cm
- Effectual Angle is <15°
- Operating frequency range is 40Hz
- Accuracy is 3mm

Connections

There are four pins on the ultrasound module that are connected to the Raspberry:

– VCC to Pin 2(VCC)
– GND to Pin 6(GND)
– TRIG to Pin12(GPIO18)
– Connect the 330Ω resistor to ECHO.

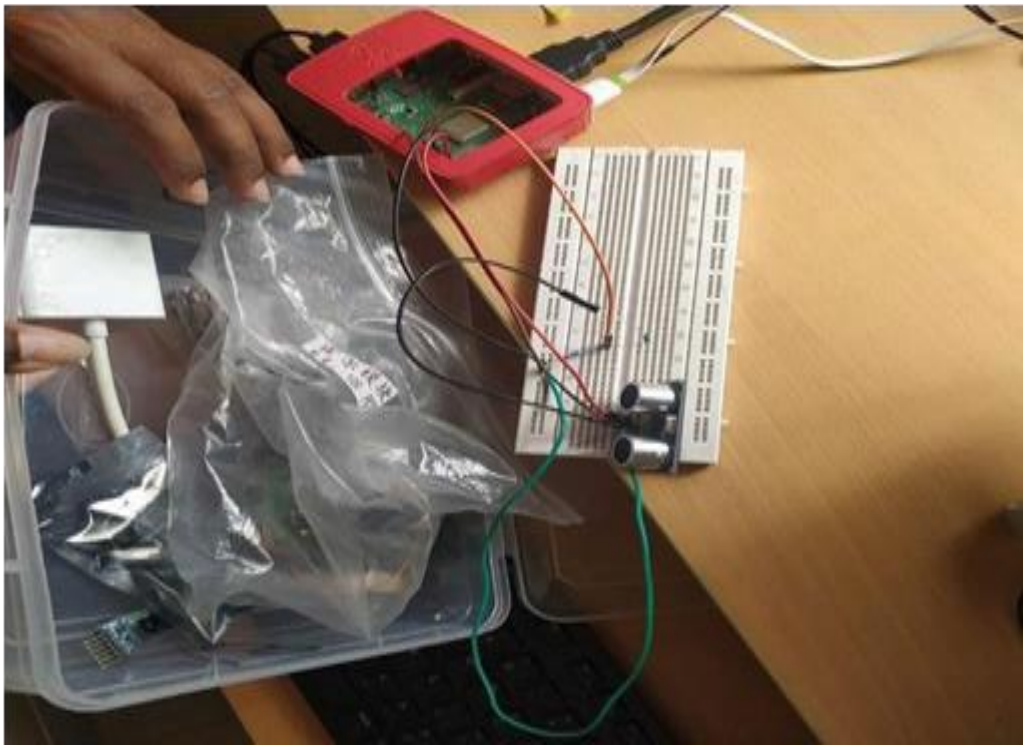
On it send you connect it to Pin18 (GPIO24) and through a 470Ω resistor you connect it also to Pin6(GND).

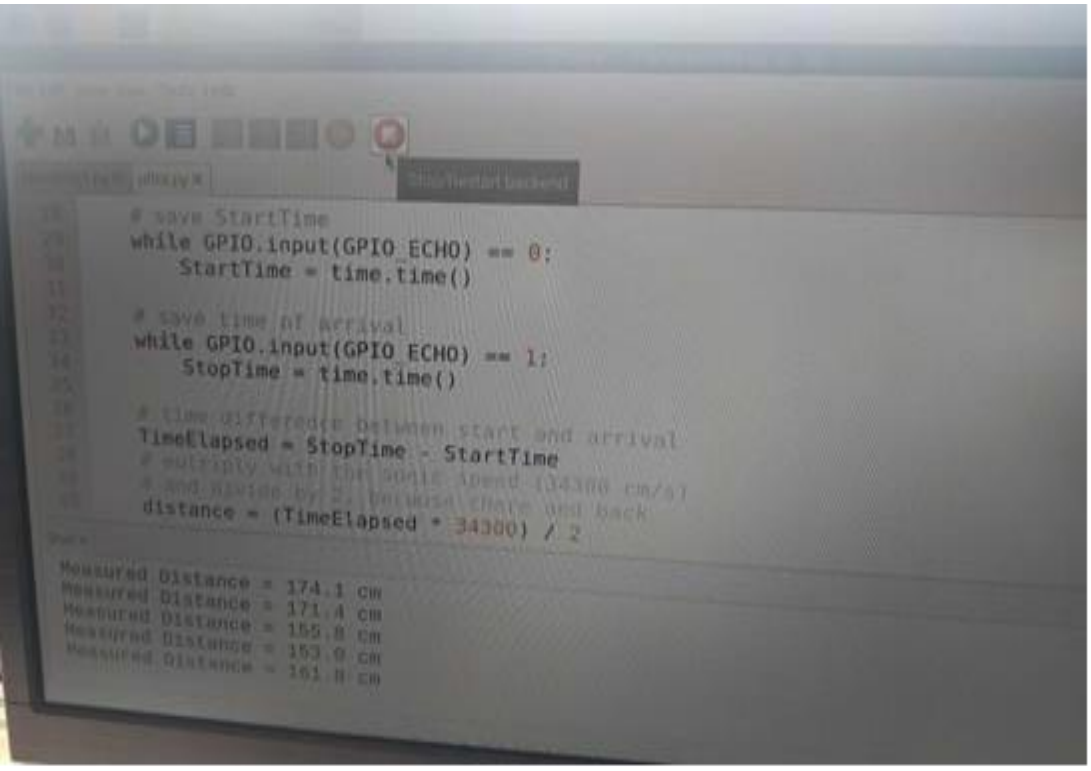
• Program:

```
import RPi.GPIO as GPIO import time #GPIOMode(BOARD/ BCM)
GPIO.setmode(GPIO.BCM)#set GPIO Pins GPIO_TRIGGER=18
GPIO_ECHO=24
#set GPIO direction (IN / OUT) GPIO.setup(GPIO_TRIGGER,GPIO.OUT)
GPIO.setup(GPIO_ECHO,GPIO.IN)
def distance():# set Trigger to HIGH GPIO.output(GPIO_TRIGGER,True) # set Trigger to HIGH
GPIO.output(GPIO_TRIGGER,True)

# set Trigger after 0.01ms to LOW
time.sleep(0.00001) GPIO.output(GPIO_TRIGGER,False) StartTime = time.time() StopTime=time.time()
# save StartTime
while GPIO.input(GPIO_ECHO) == 0: StartTime=time.time()
# save time of arrival
while GPIO.input(GPIO_ECHO) == 1: StopTime=time.time()
# time difference between start and arrival TimeElapsed=StopTime-StartTime #multiply with the
sonicspeed(34300cm/s) #and divide by2, because there and back
distance = (TimeElapsed * 34300) / 2 return distance
if name== 'main':
try:
while True:
dist=distance()
print ("Measured Distance = %.1f cm" % dist) time.sleep(1) # Reset by pressing CTRL + C
except: Keyboard Interrupt:
print("Measurement stopped by User") GPIO.cleanup()
```

OUTPUT:




while GPIO.input(GPIO_ECHO) == 0:
StartTime = time.time()
while GPIO.input(GPIO_ECHO) == 1:
StopTime = time.time()
TimeElapsed = StopTime - StartTime
distance = (TimeElapsed * 34300) / 2
Measured Distance = 174.1 cm
Measured Distance = 171.4 cm
Measured Distance = 155.8 cm
Measured Distance = 153.9 cm
Measured Distance = 161.8 cm

Interfacing PIR Sensor with Raspberry Pi

PIR SENSOR

•A passive infrared sensor (PIR sensor) is an electronic sensor that measures infrared (IR) light radiating from objects in its field of view.(range)

•PIR sensors are commonly used in security alarms and automatic lighting applications.

•PIR sensors detect general movement, but do not give information on who or what moved.

PIN DESCRIPTION

•Pin1 corresponds to the drain terminal of the device, which connected to the positive supply 5V DC.

•Pin2 corresponds to the source terminal of the device, which connects to the ground terminal via a 100K or 47K resistor. The Pin2 is the output pin of the sensor.

•Pin3 of the sensor connected to the ground.

PIR WORKING

•The PIR sensor itself has two slots in it, each slot is made of a special material that is sensitive to IR.

When the sensor is idle, both slots detect the same amount of IR, the ambient amount radiated from the room or walls or outdoors.

•When a warm body like a human or animal passes by, it first intercepts one half of the PIR sensor, which causes a positive differential change between the two halves. When the warm body leaves the sensing area, the reverse happens, whereby the sensor generates a negative differential change. These change pulses are what is detected.

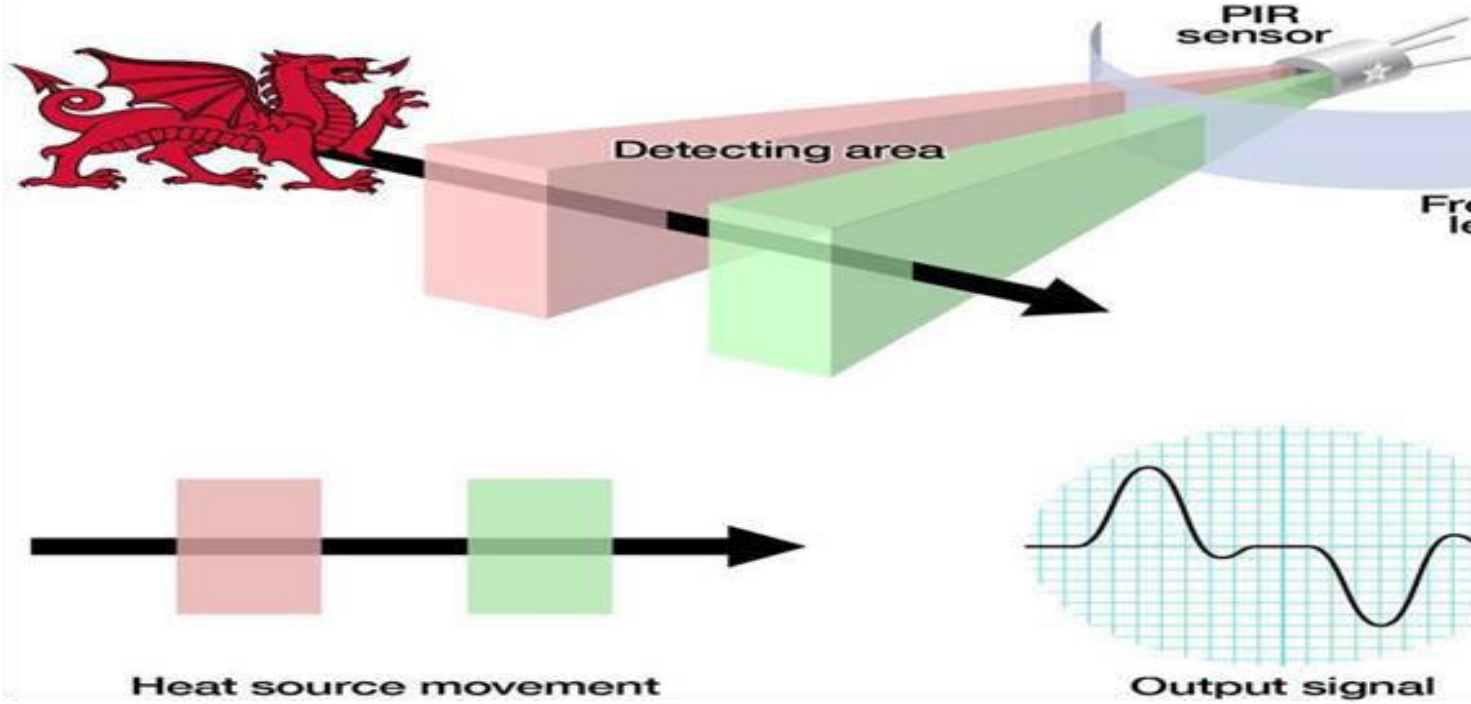
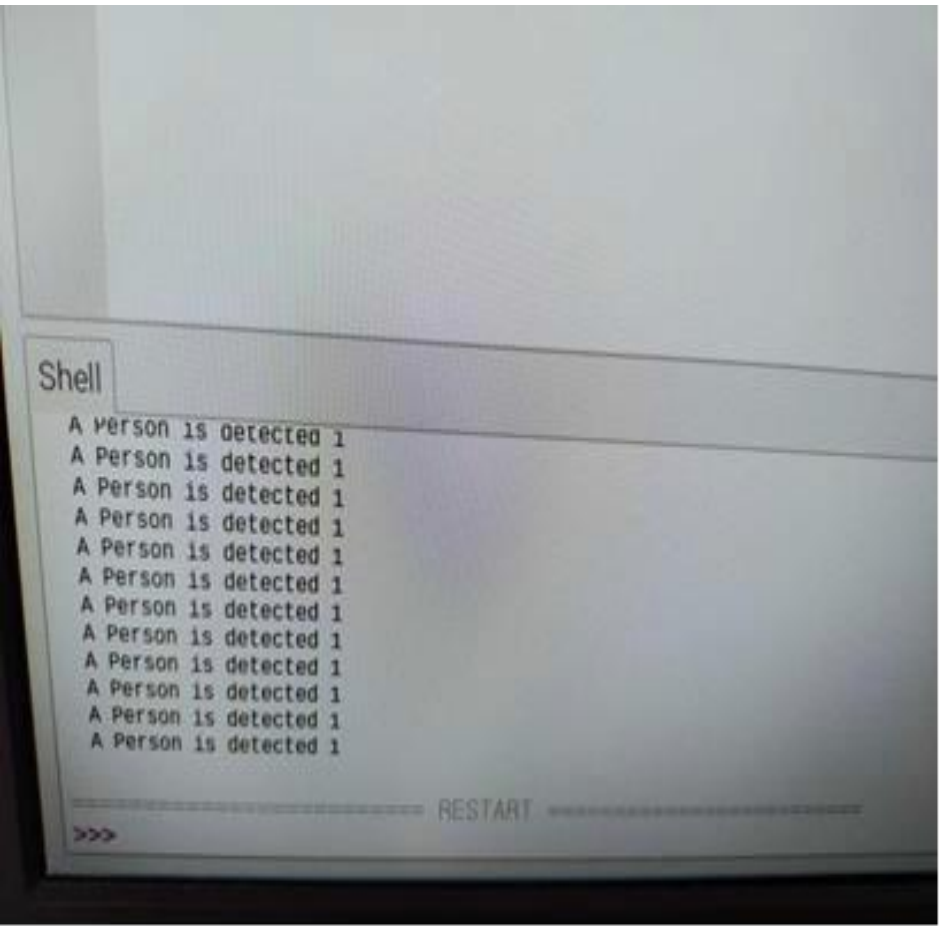


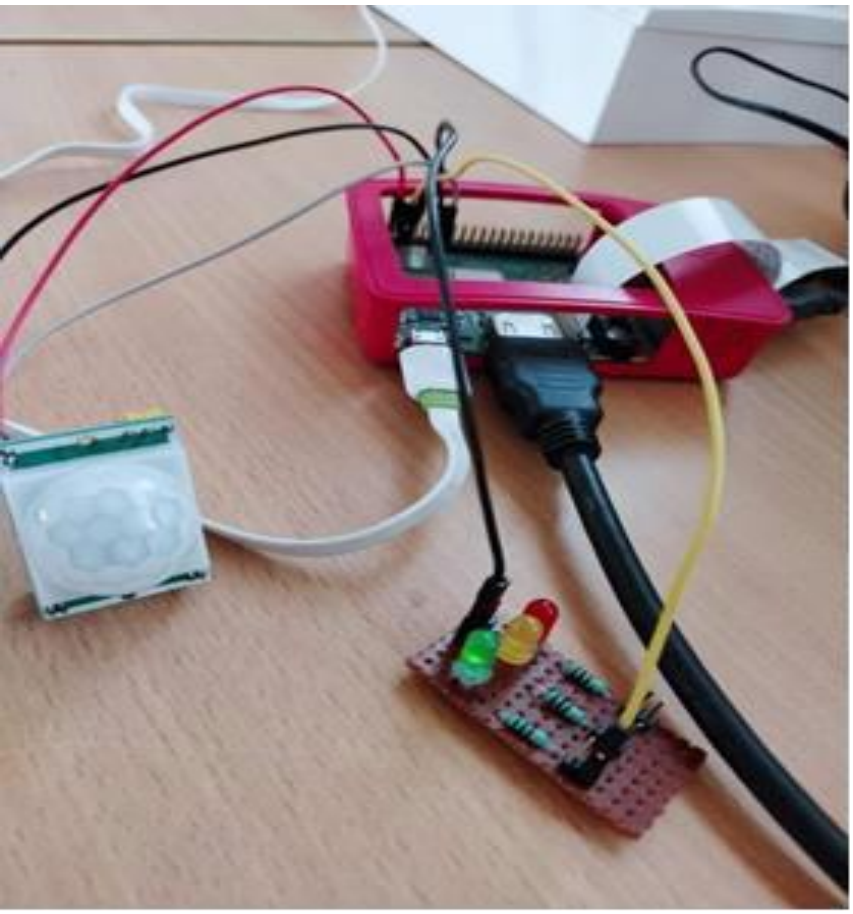
Figure: illustration of PIR working

Program

```
import RPi.GPIO as GPIO import time GPIO.setwarnings(False) GPIO.setmode(GPIO.BOARD)
GPIO.setup(11, GPIO.IN)        #Read ouput from PIR motion sensor GPIO.setup(3, GPIO.OUT)
      #LED output pin
While True:
i=GPIO.input(11)
if i == 0:#When output from motion sensor is LOW print ("No Person
detected",i)
GPIO.output(3,0) time.sleep(0.1)
elif i == 1:  #When output from motion sensor is HIGH print ("A Person is
detected",i)
GPIO.output(3,1)  #Turn ON LED time.sleep(0.1)
```

OUTPUT





Interfacing DHT22 Sensor with Raspberry Pi

DHT22: Digital Humidity Temperature sensor

The DHT-22 (also named as AM2302) is a digital-output relative humidity and temperature sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, and spits out a digital signal on the data pin.

Pin Description:

It has only four pins.

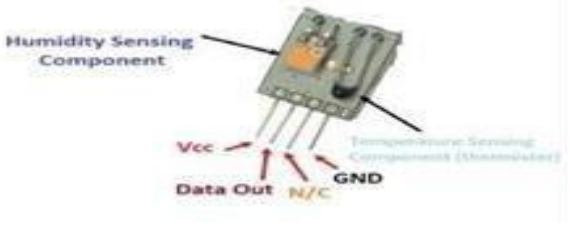
- Vcc is the power pin. Apply voltage in a range of 3.5 V to 5.0 V at this pin.
- Data Out is the digital output pin. It sends out the value of measured temperature

and humidity in the form of serial data.

- N/C is a not connect pin.
- GND: Connect the GND pin to main ground.

Connections

DHT22 pins	
1	VCC
2	DATA
3	NC
4	GND



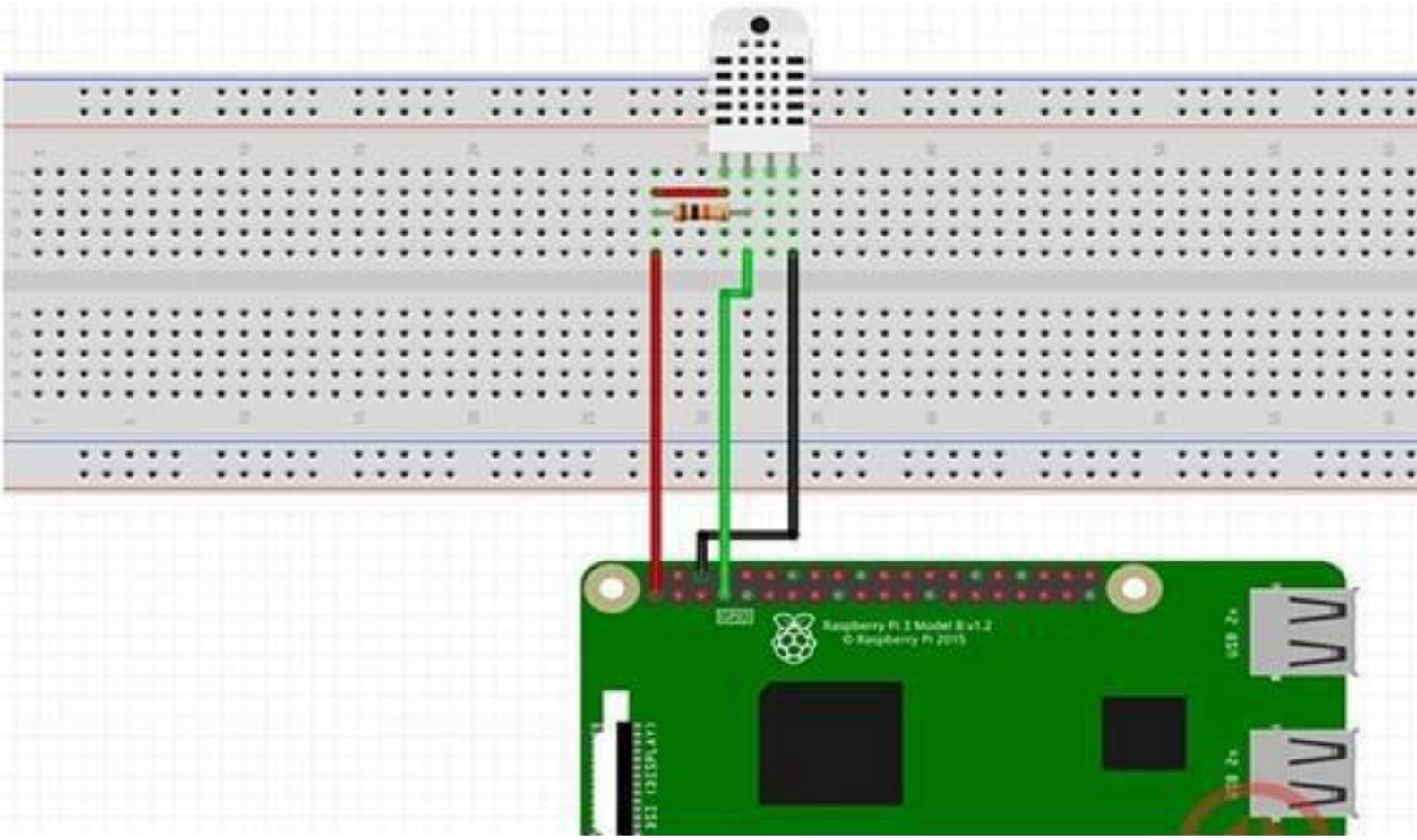
PRE REQUISTE library Installations:

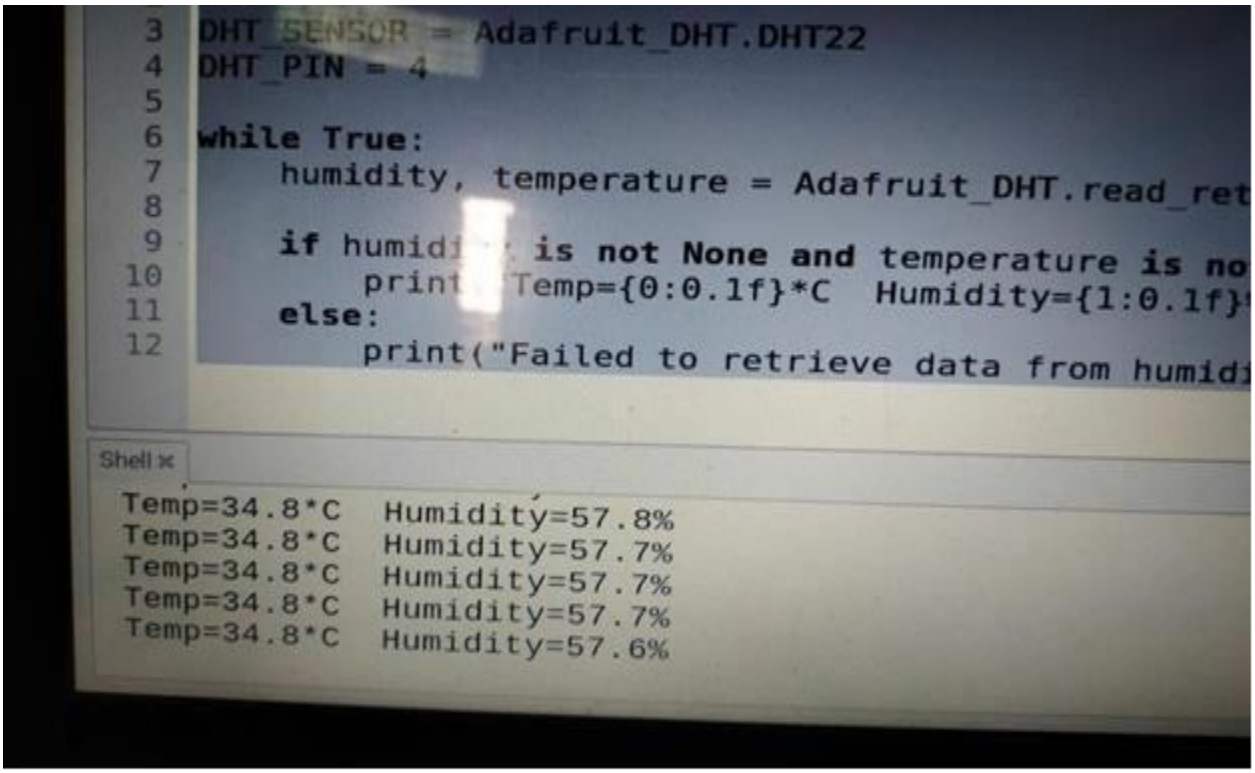
1. update both the package list and installed packages.
2. install Adafruit's DHT library to the Raspberry Pi.

• Run the following command to install the DHT library to your Raspberry Pi.

program:

```
import Adafruit_DHT
DHT_SENSOR =Adafruit_DHT.DHT22 DHT_PIN = 4
while True:
humidity, temperature = Adafruit_DHT.read_retry(DHT_SENSOR, DHT_PIN)
if humidity is not None and temperature is not None:
print("Temp={0:0.1f}*C  Humidity={1:0.1f}%".format(temperature,  humidity))
else:
print("Failed to retrieve data from sensor")
```





Interfacing PI camera with Raspberry Pi

PI camera

•The Pi camera module is a portable light weight camera that supports Raspberry Pi. It communicates with Pi using the MIPI camera serial

interface protocol. It is normally used in image processing, machine learning

or in surveillance projects.

•Pi Camera module can be used to take pictures and high definition video. Raspberry Pi Board has CSI (Camera Serial Interface) interface to

which we can attach Pi Camera module directly using 15-pin ribbon cable.

PI camera

PROGRAM 1

#capture & modify the image from picamera import PiCamera from time import sleep

from picamera import Color camera = PiCamera() camera.rotation = 180 camera.start_preview()

camera.annotate_background = Color('yellow') camera.annotate_foreground = Color('blue') camera.annotate_text_size = 80 camera.annotate_text = "Hello KMIT"

for i in range(5): sleep(5) camera.capture('/home/pi/Desktop/Rasp1%s.jpg' % i) camera.stop_preview()

PROGRAM 2

#video saving

from picamera import PiCamera from time import sleep

from picamera import PiCamera, Color camera = PiCamera()

camera.rotation = 180

camera.start_preview() camera.annotate_background = Color('yellow') camera.annotate_foreground = Color('blue') camera.annotate_text_size = 80

camera.annotate_text = " KMIT IT " camera.start_recording('/home/pi/Desktop/video.h264') sleep(5) camera.stop_recording() camera.stop_preview()